

**REPORT**  
**on**  
**Master Coding and Competitive Programming - Part-2**  
*(Course Code: V23CSSE04)*  
**B.TECH VI Semester (2023 - 2027 Batch )**  
**Academic Year: 2025-26**



**SRI VASAVI ENGINEERING COLLEGE (Autonomous)**  
**Pedatadepalli , Tadepalligudem - 534101**  
**Department of Computer Science & Engineering (Accredited by NBA)**

**SRI VASAVI ENGINEERING COLLEGE (Autonomous)**  
**PEDATADEPALLI, TADEPALLIGUDEM.**



## **Certificate**

This is to certify Mr. **Jaideep Vantipalli** bearing Roll No. **23A81A0562** of CSE Branch of VI Semester submitted the Report on \_\_\_\_\_ as a part of **Master Coding and Competitive Programming - Part-2** (Course Code: **V23CSSE04**) during the academic year 2025-2026.

**Faculty In charge**

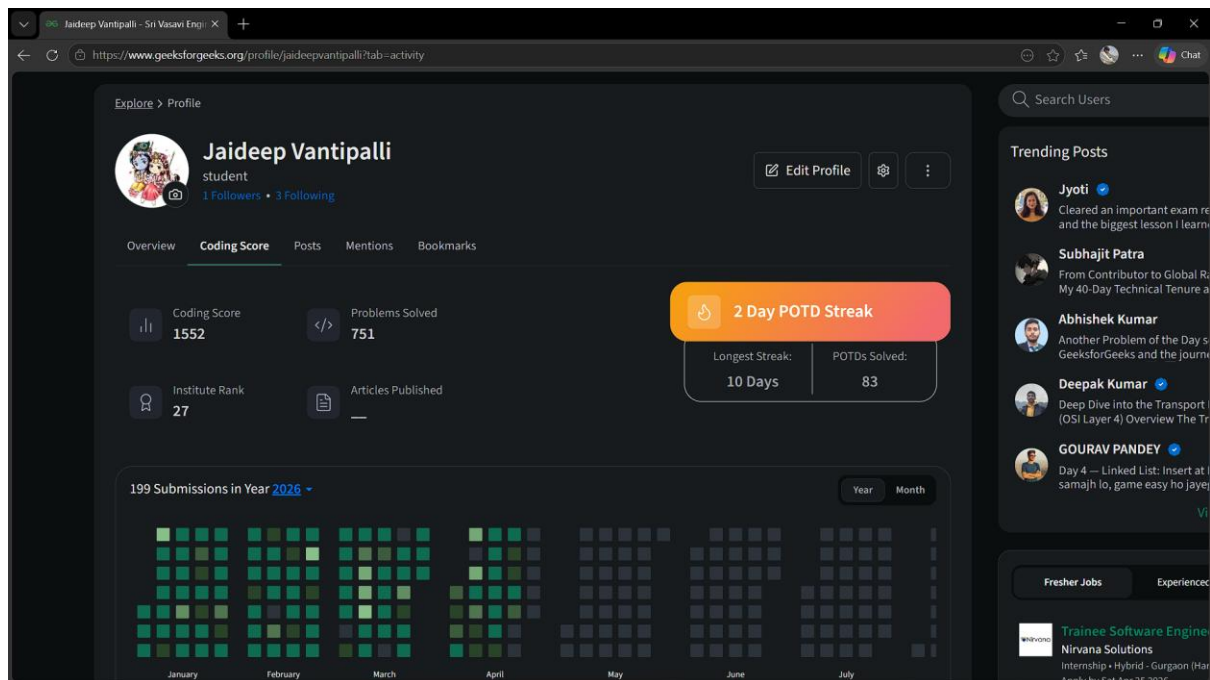
**Head of the Department**

**Examiner1**

**Examiner2**

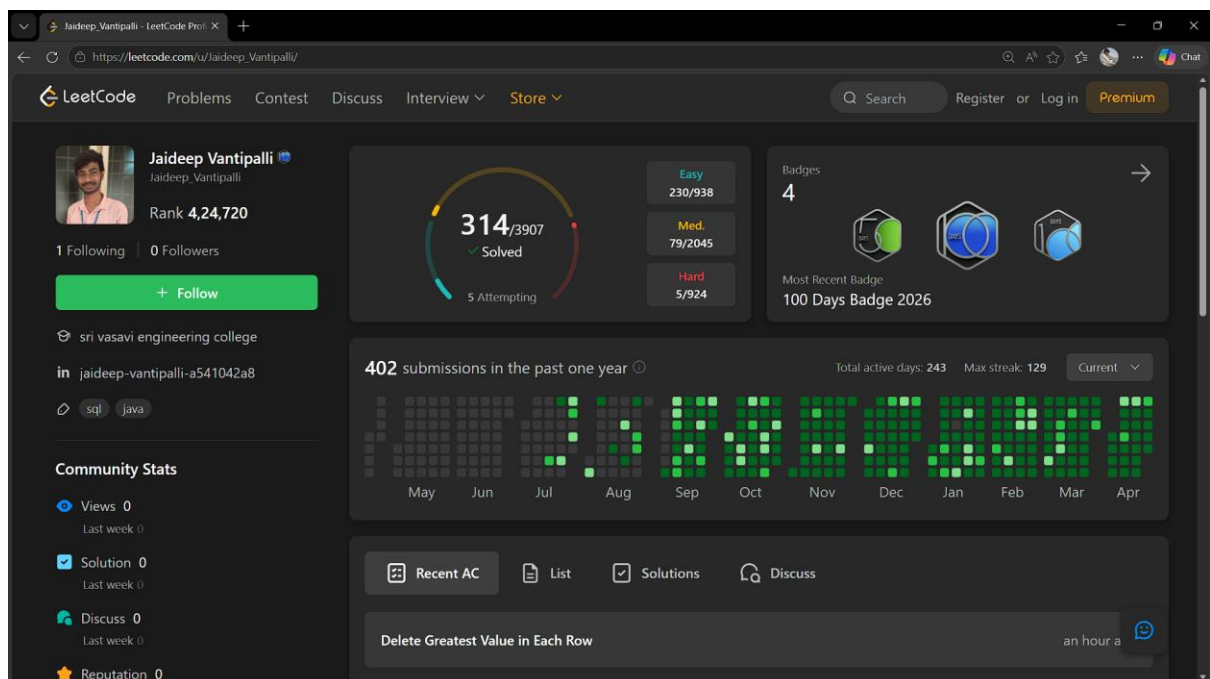
## Geeks For Geeks Profile :

<https://www.geeksforgeeks.org/user/jaideepvantipalli/>



## LeetCode Profile :

[https://leetcode.com/u/Jaideep\\_Vantipalli/](https://leetcode.com/u/Jaideep_Vantipalli/)



## INDEX

| <b>SI . NO</b>       | <b>Topics</b>                     | <b>Page No.</b> |
|----------------------|-----------------------------------|-----------------|
| <b>1</b>             | Programs on Binary Search         | 5-9             |
| <b>2</b>             | Programs on Sorting               | 10-13           |
| <b>3</b>             | Programs on TwoPointer            | 14-18           |
| <b>4</b>             | Programs on Sliding Window        | 19-26           |
| <b>5</b>             | Programs on Greedy                | 27-34           |
| <b>6</b>             | Programs on Dynamic programming   | 35-43           |
| <b>7</b>             | Programs on Linkedlist            | 44-51           |
| <b>8</b>             | Programs on Stack                 | 52-56           |
| <b>9</b>             | Programs on Queue                 | 57-61           |
| <b>10</b>            | Programs on Binary Search Tree    | 62-69           |
| <b>System Design</b> |                                   |                 |
| <b>11</b>            | Case Study : Online Shopping Cart | 70-80           |

## 1. Programs on BinarySearch.

### 1.1 First 1 in a Sorted Binary Array

**Aim :** Given a sorted array `arr` consisting of 0s and 1s. The task is to find the index (0-based indexing) of the first 1 in the given array.

```
class Solution {
    public long firstIndex(int arr[]) {
        int ans = -1;
        int low = 0;
        int high = arr.length-1;
        while(low<=high){
            int mid = (low+high)/2;
            if(arr[0] == 1){
                return 0;
            }
            if(arr[mid] == 1 && arr[mid] > arr[mid-1]){
                return mid;
            }
            else if(arr[mid] < 1){
                low = mid +1;
            }
            else{
                high = mid -1;
            }
        }
        return ans;
    }
}
```

**Output :**

The screenshot shows the GeeksforGeeks problem page for "First 1 in a Sorted Binary Array". The problem description is: "Given a sorted array `arr` consisting of 0s and 1s. The task is to find the index (0-based indexing) of the first 1 in the given array." An example input is `arr = [0, 0, 0, 0, 0, 1, 1, 1, 1]` and the output is 6. The solution code is shown in the editor, and the output window displays the result 6. The compilation results show "Compilation Completed" for Case 1.

## 1.2 Bitonic Point

**Aim :** Given an array of integers `arr[]` that is first strictly increasing and then maybe strictly decreasing, find the bitonic point, that is the maximum element in the array.

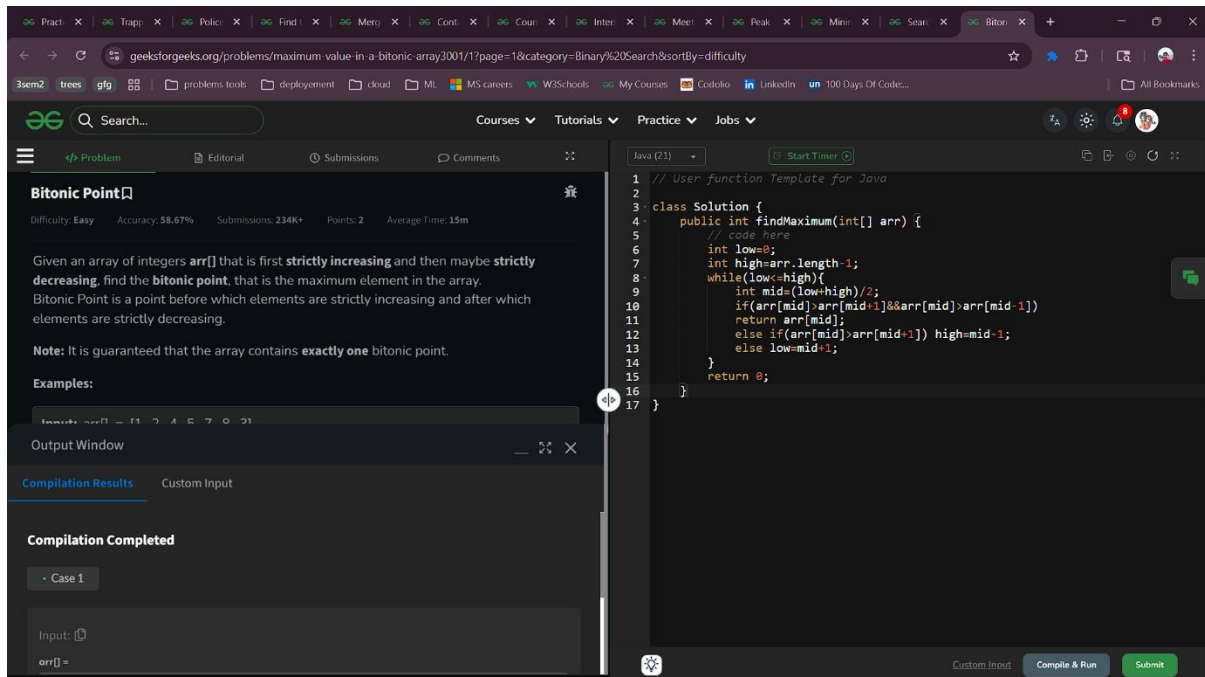
**Bitonic Point** is a point before which elements are strictly increasing and after which elements are strictly decreasing.

**Note:** It is guaranteed that the array contains exactly one bitonic point

**Code :**

```
class Solution {
    public int findMaximum(int[] arr) {
        int low=0;
        int high=arr.length-1;
        while(low<=high){
            int mid=(low+high)/2;
            if(arr[mid]>arr[mid+1]&&arr[mid]>arr[mid-1])
                return arr[mid];
            else if(arr[mid]>arr[mid+1]) high=mid-1;
            else low=mid+1;
        }
        return 0;
    }
}
```

**Output :**



### 1.3 Search in a row-wise sorted matrix

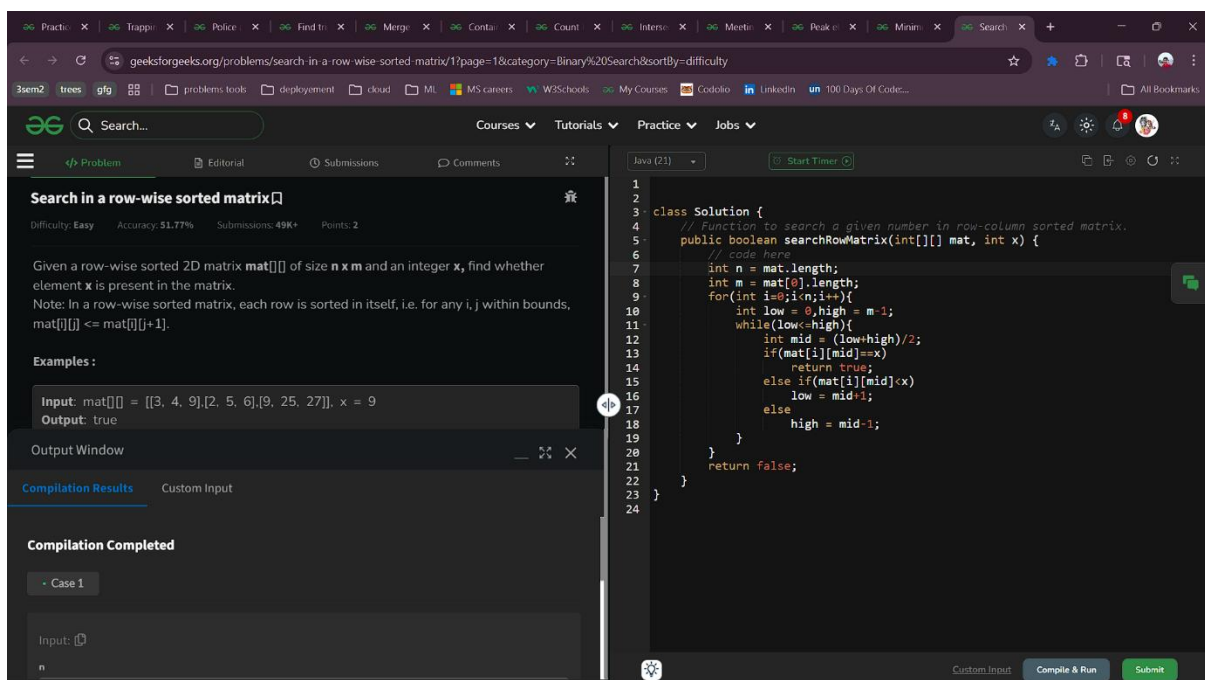
**Aim :** Given a row-wise sorted 2D matrix `mat[][]` of size `n x m` and an integer `x`, find whether element `x` is present in the matrix.

**Note:** In a row-wise sorted matrix, each row is sorted in itself, i.e. for any `i, j` within bounds, `mat[i][j] <= mat[i][j+1]`

**Code :**

```
class Solution {
    public boolean searchRowMatrix(int[][] mat, int x) {
        int n = mat.length;
        int m = mat[0].length;
        for(int i=0;i<n;i++){
            int low = 0,high = m-1;
            while(low<=high){
                int mid = (low+high)/2;
                if(mat[i][mid]==x)
                    return true;
                else if(mat[i][mid]<x)
                    low = mid+1;
                else
                    high = mid-1;
            }
        }
        return false;
    }
}
```

**Output :**



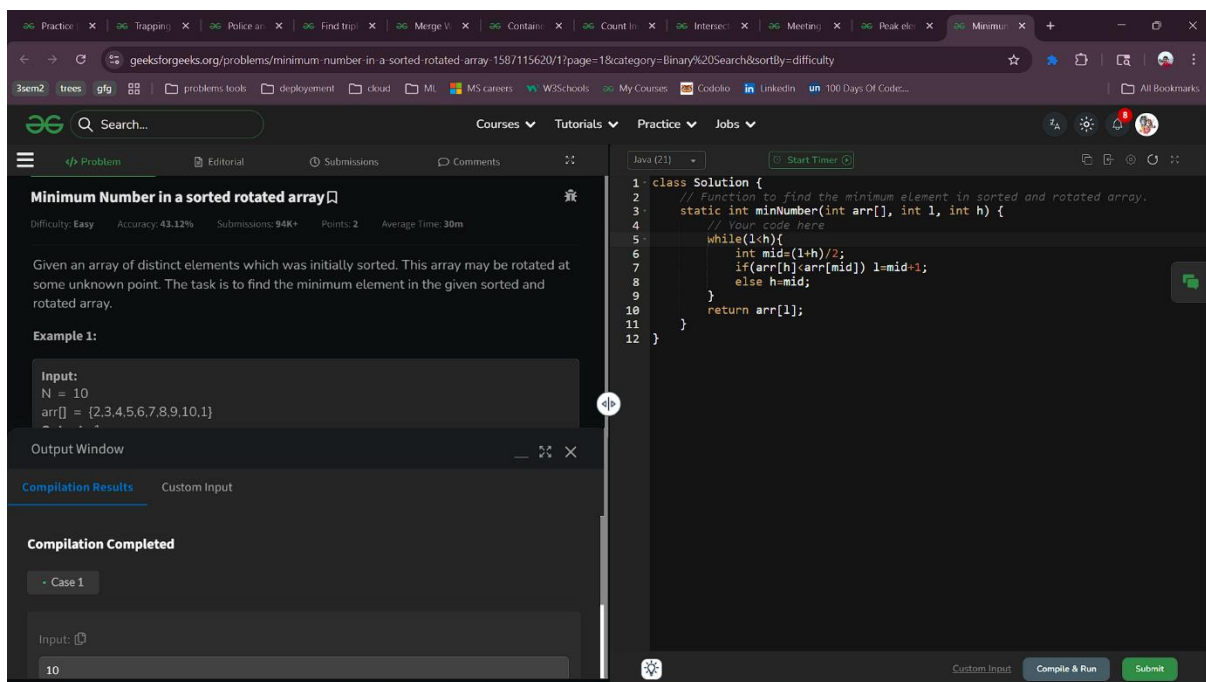
## 1.4 Minimum Number in a sorted rotated array

**Aim :** Given an array of distinct elements which was initially sorted. This array may be rotated at some unknown point. The task is to find the minimum element in the given sorted and rotated array.

**Code :**

```
class Solution {
    static int minNumber(int arr[], int l, int h) {
        // Your code here
        while(l<h){
            int mid=(l+h)/2;
            if(arr[h]<arr[mid]) l=mid+1;
            else h=mid;
        }
        return arr[l];
    }
}
```

**Output :**



## 1.5 Peak element

**Aim :** Given You are given an array `arr[]` where no two adjacent elements are same, find the index of a peak element. An element is considered to be a peak if it is greater than its adjacent elements (if they exist).

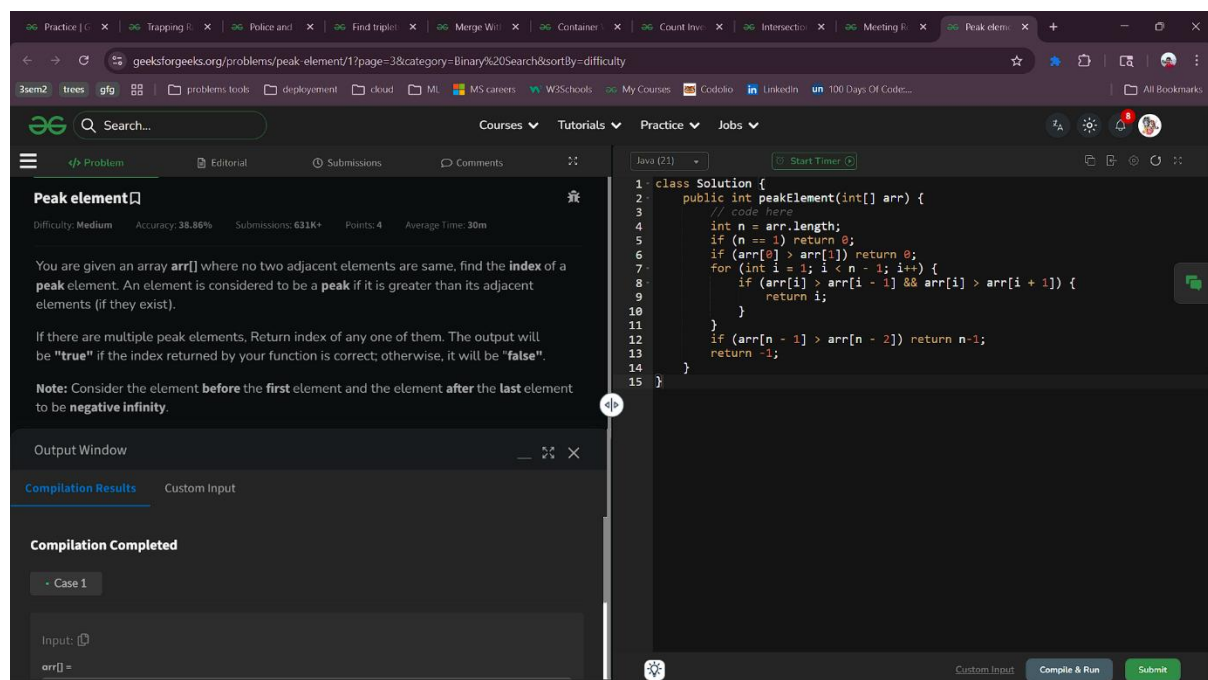
If there are multiple peak elements, Return index of any one of them. The output will be "true" if the index returned by your function is correct; otherwise, it will be "false".

**Note:** Consider the element before the first element and the element after the last element to be negative infinity

**Code :**

```
class Solution {
    public int peakElement(int[] arr) {
        int n = arr.length;
        if (n == 1) return 0;
        if (arr[0] > arr[1]) return 0;
        for (int i = 1; i < n - 1; i++) {
            if (arr[i] > arr[i - 1] && arr[i] > arr[i + 1]) {
                return i;
            }
        }
        if (arr[n - 1] > arr[n - 2]) return n-1;
        return -1;
    }
}
```

**Output :**



## 2.1 Meeting Rooms

**Aim :** Given a 2D array `arr[i][1]`, where `arr[i][0]` is the starting time of  $i^{\text{th}}$  meeting and `arr[i][1]` is the ending time of  $i^{\text{th}}$  meeting, the task is to check if it is possible for a person to attend all the meetings such that he can attend only one meeting at a particular time.

**Note:** A person can attend a meeting if its starting time is greater than or equal to the previous meeting's ending time.

**Code :**

```
class Solution {
    static boolean canAttend(int[][] arr) {
        // code here
        Arrays.sort(arr, (a, b) -> a[0] - b[0]);
        for(int i=0;i<arr.length-1;i++){
            if(arr[i][1]>arr[i+1][0])
                return false;
        }
        return true;
    }
}
```

**Output :**

The screenshot shows the 'Meeting Rooms' problem on GeeksforGeeks. The problem description states: Given a 2D array `arr[i][1]`, where `arr[i][0]` is the starting time of  $i^{\text{th}}$  meeting and `arr[i][1]` is the ending time of  $i^{\text{th}}$  meeting, the task is to check if it is possible for a person to attend all the meetings such that he can attend only one meeting at a particular time. A note specifies: A person can attend a meeting if its starting time is greater than or equal to the previous meeting's ending time.

The examples section shows an input array `arr = [[1, 4], [2, 3], [3, 4]]` and the expected output is `3`.

The solution code is as follows:

```
class Solution {
    static boolean canAttend(int[][] arr) {
        // code here
        Arrays.sort(arr, (a, b) -> a[0] - b[0]);
        for(int i=0;i<arr.length-1;i++){
            if(arr[i][1]>arr[i+1][0])
                return false;
        }
        return true;
    }
}
```

The 'Compilation Results' section shows the output for the test case: `3`.

## 2.2 Intersection of two sorted arrays

**Aim :** Given two sorted arrays `arr1[]` and `arr2[]`. Your task is to return the intersection of both arrays.

Intersection of two arrays is said to be elements that are common in both arrays. The intersection should not count duplicate elements.

**Note:** If there is no intersection then return an empty array.

**Code :**

```
class Solution {
    public ArrayList<Integer> intersection(int arr1[], int arr2[]) {
        ArrayList<Integer> list = new ArrayList<>();
        HashSet<Integer> set = new HashSet<>();
        for(int num:arr1){
            set.add(num);
        }
        for(int num: arr2){
            if(set.contains(num)){
                list.add(num);
                set.remove(num);
            }
        }
        return list;
    }
}
```

**Output ;**

The screenshot displays the GeeksforGeeks problem page for "Intersection of two sorted arrays". The problem description and examples are visible on the left. The code editor on the right contains the Java solution. The "Compilation Results" section at the bottom left shows that the code has been successfully compiled for Case 1.

### 2.3 Count Inversions

**Aim :** Given an array of integers arr[]. You have to find the Inversion Count of the array.

**Note :** Inversion count is the number of pairs of elements (i, j) such that  $i < j$  and  $arr[i] > arr[j]$ .

**Code :**

```
class Solution {
    static int inversionCount(int arr[]) {
        // Code Here
        int invcount = 0;
        invcount=mergeSort(arr,0,arr.length-1);
        return invcount;
    }
    static int mergeSort(int[] arr, int left, int right) {
        int count = 0;
        if (left<right) {
            int mid=left+(right-left)/2;
            count += mergeSort(arr,left,mid);
            count += mergeSort(arr,mid+1,right);
            count += merge(arr,left,mid,right);
        }
        return count;
    }
    static int merge(int[] arr, int left, int mid, int right) {
        int n1=mid-left+1;
        int n2=right-mid;
        int[] L = new int[n1];
        int[] R = new int[n2];
        for (int i = 0; i < n1; i++)
            L[i] = arr[left + i];
        for (int j = 0; j < n2; j++)
            R[j] = arr[mid + 1 + j];
        int i=0,j=0,k=left;
        int invcount = 0;
```

```

while (i<n1&& j<n2) {
    if (L[i]<=R[j]) {
        arr[k++]=L[i++];
    } else {
        arr[k++]=R[j++];
        invcount+=(n1-i);
    }
}

while (i < n1)
    arr[k++] = L[i++];

while (j < n2)
    arr[k++] = R[j++];

return invcount;
}
}

```

**Output :**

The screenshot displays the 'Count Inversions' problem on GeeksforGeeks. The problem description states: "Given an array of integers `arr[]`. You have to find the **Inversion Count** of the array. **Note:** Inversion count is the number of pairs of elements  $(i, j)$  such that  $i < j$  and  $arr[i] > arr[j]$ ." Examples provided are: Input: `arr[] = [2, 4, 1, 3, 5]`, Output: 3, Explanation: The sequence 2, 4, 1, 3, 5 has three inversions (2, 1), (4, 1), (4, 3).

The code editor shows a Java solution using a merge sort approach. The code is as follows:

```

12     int mid=left+(right-left)/2;
13     count += mergeSort(arr, left, mid);
14     count += mergeSort(arr, mid+1, right);
15     count += merge(arr, left, mid, right);
16 }
17 return count;
18 }
19 static int merge(int[] arr, int left, int mid, int right) {
20     int n1=mid-left+1;
21     int n2=right-mid;
22     int[] L = new int[n1];
23     int[] R = new int[n2];
24     for (int i = 0; i < n1; i++)
25         L[i] = arr[left + i];
26     for (int j = 0; j < n2; j++)
27         R[j] = arr[mid + 1 + j];
28     int i=0, j=0, k=left;
29     int invcount = 0;
30     while (i<n1&&j<n2) {
31         if (L[i]<=R[j]) {
32             arr[k++]=L[i++];
33         } else {
34             arr[k++]=R[j++];
35             invcount+=(n1-i);
36         }
37     }
38     while (i < n1)
39         arr[k++] = L[i++];
40     while (j < n2)
41         arr[k++] = R[j++];
42     return invcount;
43 }
44 }
45 }

```

The output window shows the result for the input array [2, 4, 1, 3, 5]:

```

Input: arr[] = [2, 4, 1, 3, 5]
Output: 3
Explanation: The sequence 2, 4, 1, 3, 5 has three inversions (2, 1), (4, 1), (4, 3).

```

### 3.1 Container With Most Water

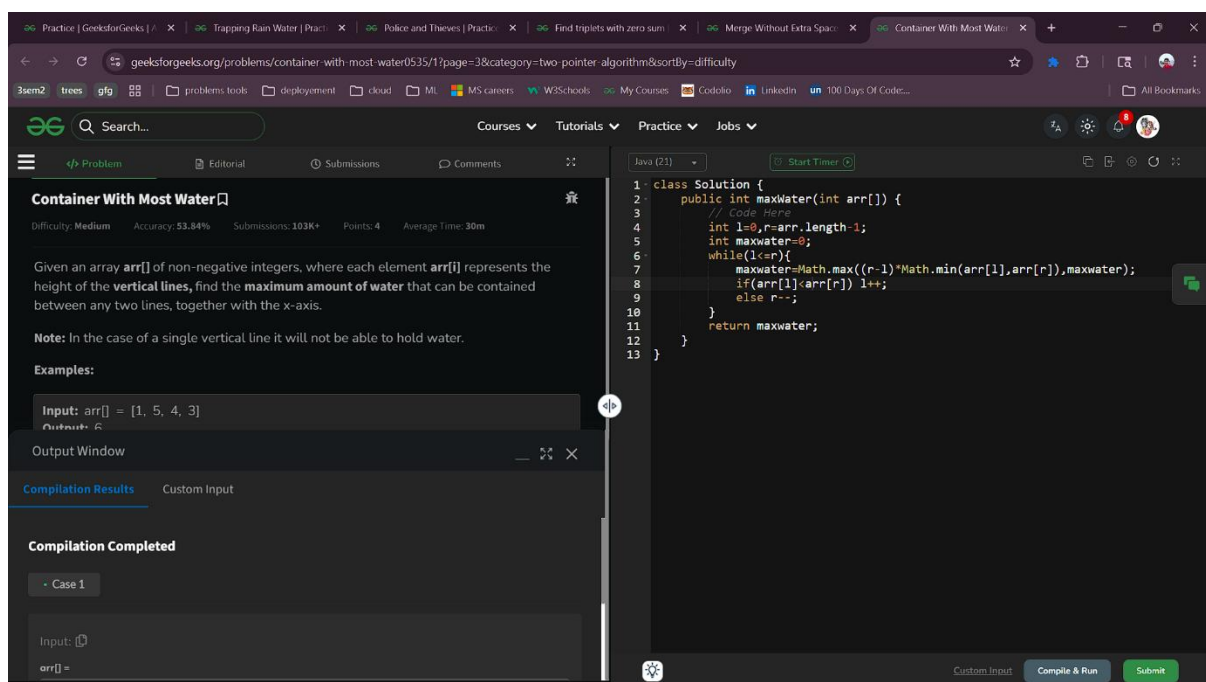
**Aim :** Given an array `arr[]` of non-negative integers, where each element `arr[i]` represents the height of the vertical lines, find the maximum amount of water that can be contained between any two lines, together with the x-axis.

**Note:** In the case of a single vertical line it will not be able to hold water

**Code :**

```
class Solution {
    public int maxWater(int arr[]) {
        // Code Here
        int l=0,r=arr.length-1;
        int maxwater=0;
        while(l<=r){
            maxwater=Math.max((r-l)*Math.min(arr[l],arr[r]),maxwater);
            if(arr[l]<arr[r]) l++;
            else r--;
        }
        return maxwater;
    }
}
```

**Output :**



### 3.2 Merge Without Extra Space

**Aim :** Given two sorted arrays `a[]` and `b[]` of size `n` and `m` respectively, the task is to merge them in sorted order without using any extra space. Modify `a[]` so that it contains the first `n` elements and modify `b[]` so that it contains the last `m` elements.

**Code :**

```
class Solution {
    public void mergeArrays(int a[], int b[]) {
        // code here
        int i=a.length-1;
        int j=0;
        while(i>=0 && j<b.length)
        {
            if(a[i]>b[j])
            {
                int temp=a[i];
                a[i]=b[j];
                b[j]=temp;
                i--;
                j++;
            }
            else j++;
        }
        Arrays.sort(a);
        Arrays.sort(b);
    }
}
```

**Output :**

The screenshot displays the GeeksforGeeks problem page for 'Merge Without Extra Space'. It includes the problem statement, examples, and a code editor with the following Java code:

```
1 class Solution {
2     public void mergeArrays(int a[], int b[]) {
3         // code here
4         int i=a.length-1;
5         int j=0;
6         while(i>=0 && j<b.length)
7         {
8             if(a[i]>b[j])
9             {
10                int temp=a[i];
11                a[i]=b[j];
12                b[j]=temp;
13                i--;
14                j++;
15            }
16            else j++;
17        }
18        Arrays.sort(a);
19        Arrays.sort(b);
20    }
21 }
22
```

The 'Compilation Results' window shows 'Compilation Completed' for Case 1. The input field shows 'a[] = ' and the output field shows 'a[] = '.

### 3.3 Find triplets with zero sum

**Aim :** Given an array `arr[]` of integers, determine whether it contains a triplet whose sum equals zero. Return `true` if such a triplet exists, otherwise, return `false`.

**Code :**

```
class Solution {
    public boolean findTriplets(int[] arr) {
        // code here.
        Arrays.sort(arr);
        int n=arr.length;
        for (int i = 0;i<n;i++) {
            int l=i+1;
            int r=n-1;
            while(l<r){
                if (arr[i]+arr[l]+arr[r]==0){
                    return true;
                } else if (arr[i]+arr[l]+arr[r]<0)
                    l++;
                else
                    r--;
            }
        }
        return false;
    }
}
```

**Output :**

The screenshot displays the GeeksforGeeks IDE interface. On the left, the problem 'Find triplets with zero sum' is selected, showing its difficulty (Medium), accuracy (25.81%), and a description. An example input and output are provided. The main editor on the right contains the Java code for the solution. At the bottom, the 'Output Window' shows 'Compilation Completed' and 'Case 1' with the input 'arr ='.

### 3.4 Police and Thieves

**Aim:** Given an array `arr[]`, where each element contains either a 'P' for policeman or a 'T' for thief. Find the maximum number of thieves that can be caught by the police. Keep in mind the following conditions :

1. Each policeman can catch only one thief.
2. A policeman cannot catch a thief who is more than `k` units away from him.

**Code :**

```
class Solution {
    public int catchThieves(char[] arr, int k) {
        Queue<Integer> police=new LinkedList<>();
        Queue<Integer> thiefe=new LinkedList<>();
        int c=0;
        for(int i=0;i<arr.length;i++){
            if(arr[i]=='P') police.add(i);
            else thiefe.add(i);
            while(!police.isEmpty() && !thieve.isEmpty()){
                if(Math.abs(police.peek()-thieve.peek())<=k){
                    c++;
                    police.poll();
                    thiefe.poll();
                } else if(police.peek()<thieve.peek()) police.poll();
                else thiefe.poll();
            }
        }
        return c;
    }
}
```

**Output :**

The screenshot shows the GeeksforGeeks website interface for the 'Police and Thieves' problem. On the left, the problem description is visible, including the aim and conditions. The main area displays the Java solution code, which is identical to the code provided in the text. At the bottom, the 'Compilation Results' section shows 'Compilation Completed' for 'Case 1' with an input array `arr[] =`.

### 3.5 Trapping Rain Water

**Aim :** Given an array `arr[]` with non-negative integers representing the height of blocks. If the width of each block is 1, compute how much water can be trapped between the blocks during the rainy season. **Code :**

```
class Solution {
    public int maxWater(int arr[]) {
        int n=arr.length;
        int l=0,r=n-1;
        int lm=0,rm=0;
        int water=0;
        while(l<r){
            if(arr[l]<arr[r]){
                lm=Math.max(lm,arr[l]);
                water+=lm-arr[l];
                l++;
            }
            else{
                rm=Math.max(rm,arr[r]);
                water+=rm-arr[r];
                r--;
            }
        }
        return water;
    }
}
```

**Output :**

The screenshot shows the GeeksforGeeks website interface for the 'Trapping Rain Water' problem. The problem description states: 'Given an array `arr[]` with non-negative integers representing the height of blocks. If the width of each block is 1, compute how much water can be trapped between the blocks during the rainy season.' Examples provided: Input: `arr[] = [3, 0, 1, 0, 4, 0, 2]`, Output: 10. Explanation: Total water trapped = 0 + 3 + 2 + 3 + 0 + 2 + 0 = 10 units.

The code editor shows the following Java code:

```
1 class Solution {
2     public int maxWater(int arr[]) {
3         // code here
4         int n=arr.length;
5         int l=0,r=n-1;
6         int lm=0,rm=0;
7         int water=0;
8         while(l<r){
9             if(arr[l]<arr[r]){
10                lm=Math.max(lm,arr[l]);
11                water+=lm-arr[l];
12                l++;
13            }
14            else{
15                rm=Math.max(rm,arr[r]);
16                water+=rm-arr[r];
17                r--;
18            }
19        }
20        return water;
21    }
22 }
23
```

The output window shows the input `arr[] = 3 0 1 0 4 0 2` and the output `10`.

#### 4.1 First negative in every window of size k

**Aim :** Given an array `arr[]` and a positive integer `k`, find the first negative integer for each and every window(contiguous subarray) of size `k`.

**Note:** If a window does not contain a negative integer, then return 0 for that window

class Solution {

```
static List<Integer> firstNegInt(int arr[], int k) {
    Deque<Integer> dq = new LinkedList<>();
    List<Integer> res = new ArrayList<>();
    int n = arr.length;
    for (int i = 0; i < k; i++)
        if (arr[i] < 0) dq.addLast(i);
    if (!dq.isEmpty())
        res.add(arr[dq.peekFirst()]);
    else
        res.add(0);
    for (int i = k; i < n; i++) {
        if (arr[i] < 0)
            dq.addLast(i);

        while (!dq.isEmpty() && dq.peekFirst() < (i - k + 1))
            dq.pollFirst();
        if (!dq.isEmpty())
            res.add(arr[dq.peekFirst()]);
        else
            res.add(0);
    }
    return res;
}
```

}**Output :**

The screenshot shows a web IDE interface with the following components:

- Top Bar:** Navigation tabs for various problems and a search bar.
- Left Pane:**
  - Problem Title:** First negative in every window of size k
  - Difficulty:** Medium
  - Accuracy:** 48.61%
  - Submissions:** 232K+
  - Points:** 4
  - Average Time:** 15m
  - Description:** Given an array `arr[]` and a positive integer `k`, find the first negative integer for each and every window(contiguous subarray) of size `k`.
  - Note:** If a window does not contain a negative integer, then return 0 for that window.
  - Examples:**
    - Input: `arr[] = [-8, 2, 3, -6, 10]`, `k = 2`
    - Output: `[-8, 0, -6, -6]`
    - Explanation:
  - Output Window:** Shows the result of the compilation.
  - Compilation Results:** Shows 'Compilation Completed'.
- Right Pane:**
  - Code Editor:** Contains the Java code implementation for the problem, using a `Deque` to maintain a sliding window of indices.
  - Language:** Java (2.1)
  - Buttons:** 'Start Timer', 'Compile & Run', and 'Submit'.

## 4.2 Max Xor Subarray of size K

**Aim :** Given an array of integers `arr[]` and a number `k`. Return the maximum xor of a subarray of size `k`.

**Note:** A subarray is a contiguous part of any given array.

```
class Solution {
    public int maxSubarrayXOR(int[] arr, int k) {
        // code here
        int n = arr.length;
        int currXor = 0;
        for (int i = 0; i < k; i++) {
            currXor ^= arr[i];
        }

        int maxXor = currXor;
        for (int i = k; i < n; i++) {
            currXor ^= arr[i];
            currXor ^= arr[i - k];
            maxXor = Math.max(maxXor, currXor);
        }

        return maxXor;
    }
}
```

**Output :**

The screenshot shows a web browser with the URL `geeksforgeeks.org/problems/max-xor-subarray-of-size-k/1?page=2&category=sliding-window&sortBy=difficulty`. The page displays the problem 'Max Xor Subarray of size K' with a difficulty of Medium, accuracy of 55.63%, and 29K+ submissions. The problem description states: 'Given an array of integers `arr[]` and a number `k`. Return the maximum xor of a subarray of size `k`.' A note specifies: 'A subarray is a contiguous part of any given array.' Examples provided are: 'Input: `arr[] = [2, 5, 8, 1, 1, 3]`, `k = 3`' and 'Output: 15'. The explanation states: 'arr[0] ^ arr[1] ^ arr[2] = 15, which is maximum'. The solution code is shown in the editor, matching the code provided in the text. The 'Compilation Results' section shows 'Compilation Completed' for 'Case 1'. The 'Output Window' shows 'Input: [2, 5, 8, 1, 1, 3]' and 'arr[] = [2, 5, 8, 1, 1, 3]'.

### 4.3 K Sized Subarray Maximum

**Aim :** Given an array `arr[]` of positive integers and an integer `k`. You have to find the maximum value for each contiguous subarray of size `k`. Return an array of maximum values corresponding to each contiguous subarray.

```
class Solution {
    public ArrayList<Integer> maxOfSubarrays(int[] arr, int k) {
        ArrayList<Integer> al=new ArrayList<>();
        Deque<Integer> dq=new ArrayDeque<>();
        for(int i=0;i<k;i++){
            while(!dq.isEmpty() && arr[dq.peekLast()]<=arr[i] ){
                dq.removeLast();
            }
            dq.add(i);
        }
        al.add(arr[dq.peekFirst()]);
        for(int i=k;i<arr.length;i++){
            while(!dq.isEmpty() && dq.peekFirst()<=i-k){
                dq.removeFirst();
            }
            while(!dq.isEmpty() && arr[dq.peekLast()]<=arr[i]){
                dq.removeLast();
            }
            dq.add(i);
            al.add(arr[dq.peekFirst()]);
        }
        return al;
    }
}
```

**Output :**

The screenshot shows the GeeksforGeeks IDE interface. On the left, the problem description for 'K Sized Subarray Maximum' is visible, including the input array `arr[] = [1, 2, 3, 1, 4, 5, 2, 3, 6]` and `k = 3`. The output is `[3, 3, 4, 5, 5, 5, 6]`. The main editor on the right contains the Java code for the solution, which uses a sliding window approach with a deque to maintain the maximum element in the current window of size `k`. The code is as follows:

```
1 class Solution {
2     public ArrayList<Integer> maxOfSubarrays(int[] arr, int k) {
3         // code here
4         ArrayList<Integer> al=new ArrayList<>();
5         Deque<Integer> dq=new ArrayDeque<>();
6         for(int i=0;i<k;i++){
7             while(!dq.isEmpty() && arr[dq.peekLast()]<=arr[i] ){
8                 dq.removeLast();
9             }
10            dq.add(i);
11        }
12        al.add(arr[dq.peekFirst()]);
13        for(int i=k;i<arr.length;i++){
14            while(!dq.isEmpty() && dq.peekFirst()<=i-k){
15                dq.removeFirst();
16            }
17            while(!dq.isEmpty() && arr[dq.peekLast()]<=arr[i]){
18                dq.removeLast();
19            }
20            dq.add(i);
21            al.add(arr[dq.peekFirst()]);
22        }
23        return al;
24    }
25 }
```

At the bottom, the 'Compilation Results' section shows 'Compilation Completed' for Case 1. The input field shows `arr[] =`.

#### 4.4 Subarray with 0 sum

**Aim :** Given an array of integers, `arr[]`. Find if there is a subarray (of size at least one) with 0 sum. Return true/false depending upon whether there is a subarray present with 0-sum or not.

```
class Solution {
    // Function to check whether there is a subarray present with 0-sum or not.
    static boolean findsum(int arr[]) {
        // Your code here
        HashSet<Integer> hs=new HashSet<>();
        int psum=0;
        for(int i=0;i<arr.length;i++){
            psum+=arr[i];
            if(psum==0 || hs.contains(psum)) return true;
            hs.add(psum);
        }
        return false;
    }
}
```

**Output :**

The screenshot shows the GeeksforGeeks problem page for "Subarray with 0 sum". The problem description states: "Given an array of integers, `arr[]`. Find if there is a subarray (of size at least one) with 0 sum. Return true/false depending upon whether there is a subarray present with 0-sum or not." Examples provided are:
 

- Input: `arr[] = [4, 2, -3, 1, 6]`, Output: true, Explanation: 2, -3, 1 is the subarray with a sum of 0.
- Input: `arr = [4, 2, 0, 1, 6]`, Output: true, Explanation: 0 is one of the element in the array so there exist a subarray with sum 0.
- Input: `arr = [1, 2, -1]`, Output: false.

 Constraints:  $1 \leq \text{arr.size} \leq 10^5$  and  $-10^5 \leq \text{arr}[i] \leq 10^5$ .

The code editor on the right shows the following Java solution:

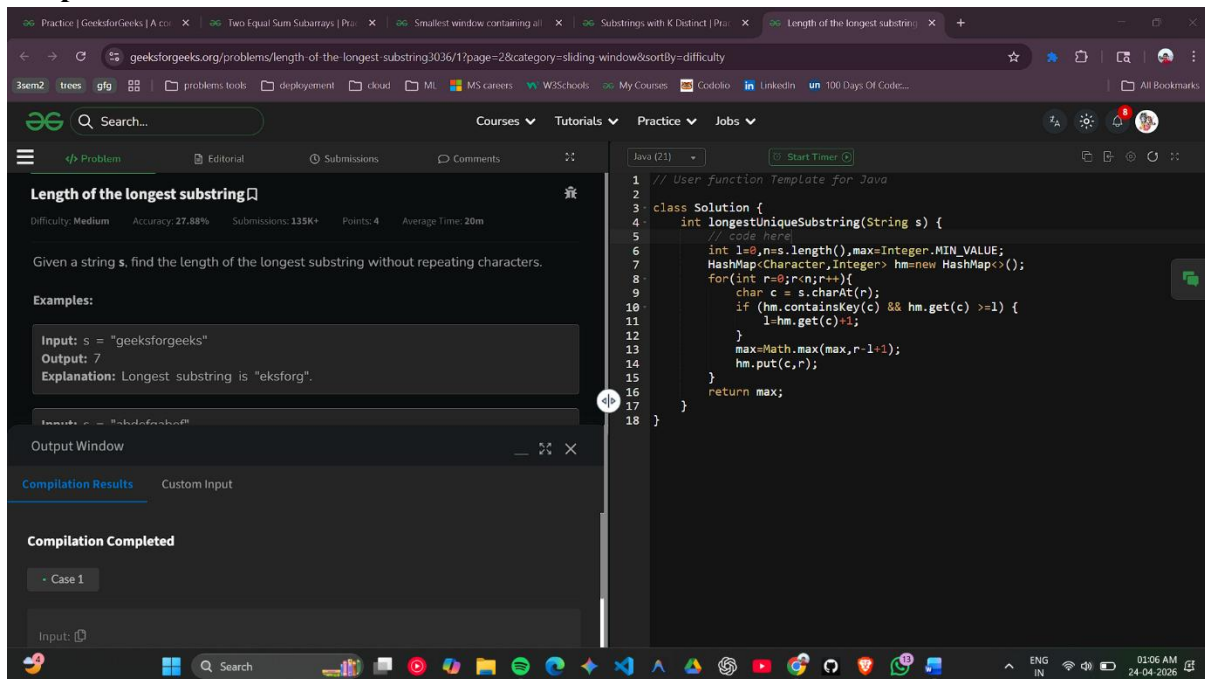
```
1 class Solution {
2     // Function to check whether there is a subarray present with 0-sum or not.
3     static boolean findsum(int arr[]) {
4         // Your code here
5         HashSet<Integer> hs=new HashSet<>();
6         int psum=0;
7         for(int i=0;i<arr.length;i++){
8             psum+=arr[i];
9             if(psum==0 || hs.contains(psum)) return true;
10            hs.add(psum);
11        }
12        return false;
13    }
14 }
```

## 4.5 Length of the longest substring

**Aim :** You Given a string *s*, find the length of the longest substring without repeating characters.

```
class Solution {
    int longestUniqueSubstring(String s) {
        int l=0,n=s.length(),max=Integer.MIN_VALUE;
        HashMap<Character,Integer> hm=new HashMap<>();
        for(int r=0;r<n;r++){
            char c = s.charAt(r);
            if (hm.containsKey(c) && hm.get(c) >=l) {
                l=hm.get(c)+1;
            }
            max=Math.max(max,r-l+1);
            hm.put(c,r);
        }
        return max;
    }
}
```

**Output :**



## 4.6 Substrings with K Distinct

**Aim :** You are given a string  $s$  consisting of lowercase characters and an integer  $k$ , You have to count all possible substrings that have exactly  $k$  distinct characters.

```
class Solution {
    static int countSubstr(String s, int k) {
        int n = s.length();
        int ans = 0;
        ans = count(s, k) - count(s, k - 1);
        return ans;
    }
    static int count(String s, int k) {
        int n = s.length();
        int ans = 0;
        int[] freq = new int[26];
        int distinctCnt = 0;
        int i = 0;
        for (int j = 0; j < n; j++) {
            freq[s.charAt(j) - 'a']++;
            if (freq[s.charAt(j) - 'a'] == 1) distinctCnt++;
            while (distinctCnt > k) {
                freq[s.charAt(i) - 'a']--;
                if (freq[s.charAt(i) - 'a'] == 0) distinctCnt--;
                i++;
            }
            ans += j - i + 1;
        }
        return ans;
    }
}
```

**Output :**

The screenshot shows the GeeksforGeeks problem page for "Substrings with K Distinct". The problem description states: "You are given a string  $s$  consisting of lowercase characters and an integer  $k$ . You have to count all possible substrings that have exactly  $k$  distinct characters." Examples provided are: Input:  $s = \text{"abc"}, k = 2$ ; Output: 2; Explanation: Possible substrings are ["ab", "bc"]. The Java solution in the editor is as follows:

```
1 class Solution {
2     static int countSubstr(String s, int k) {
3         int n = s.length();
4         int ans = 0;
5
6         ans = count(s, k) - count(s, k - 1);
7         return ans;
8     }
9     static int count(String s, int k) {
10        int n = s.length();
11        int ans = 0;
12
13        int[] freq = new int[26];
14        int distinctCnt = 0;
15        int i = 0;
16
17        for (int j = 0; j < n; j++) {
18            freq[s.charAt(j) - 'a']++;
19            if (freq[s.charAt(j) - 'a'] == 1) distinctCnt++;
20
21            while (distinctCnt > k) {
22                freq[s.charAt(i) - 'a']--;
23                if (freq[s.charAt(i) - 'a'] == 0) distinctCnt--;
24                i++;
25            }
26
27            ans += j - i + 1;
28        }
29
30        return ans;
31    }
32 }
33
34
```

## 4.7 Smallest window containing all characters

**Aim :** Given two strings *s* and *p*. Find the smallest substring in *s* consisting of all the characters (including duplicates) of the string *p*. Return empty string in case no such substring is present. If there are multiple such substring of the same length found, return the one with the least starting index.

```
class Solution {
    public static String minWindow(String s, String p) {
        if (p.length() > s.length()) return "";
        int[] freq = new int[256];
        for (char c : p.toCharArray()) {
            freq[c]++;
        }
        int left = 0, count = p.length();
        int minLen = Integer.MAX_VALUE;
        int start = 0;
        for (int right = 0; right < s.length(); right++) {
            if (freq[s.charAt(right)] > 0)
                count--;
            freq[s.charAt(right)]--;
            while (count == 0) {
                if (right - left + 1 < minLen) {
                    minLen = right - left + 1;
                    start = left;
                }
                freq[s.charAt(left)]++;
                if (freq[s.charAt(left)] > 0)
                    count++;
                left++;
            }
        }
        return minLen == Integer.MAX_VALUE ? "" : s.substring(start, start + minLen);
    }
}
```

**Output :**

The screenshot shows a web IDE interface for the problem "Smallest window containing all characters". The problem description on the left states: "Given two strings *s* and *p*. Find the smallest substring in *s* consisting of all the characters (including duplicates) of the string *p*. Return empty string in case no such substring is present. If there are multiple such substring of the same length found, return the one with the least starting index." Examples show input *s* = "timetopractice", *p* = "top" and output "top". The right side shows the Java solution code, which is a static method `minWindow` that uses a sliding window approach with a frequency array to find the smallest substring in *s* that contains all characters of *p*. The output window shows the result for the input *s* = "timetopractice", *p* = "top" is "top".

## 4.8 Two Equal Sum Subarrays

**Aim :** Given an array of integers `arr[]`, return `true` if it is possible to split it in two subarrays (without reordering the elements), such that the sum of the two subarrays are equal. If it is not possible then return `false`.

**Code :**

```
class Solution {
    public boolean canSplit(int arr[]) {
        // code here
        int lsum=0,rsum=0;
        for(int i=0;i<arr.length;i++) lsum+=arr[i];
        for(int i=arr.length-1;i>=0;i--){
            rsum+=arr[i];
            lsum-=arr[i];
            if(lsum==rsum) return true;
        }
        return false;
    }
}
```

**Output :**

The screenshot shows the GeeksforGeeks website interface. On the left, the problem 'Two Equal Sum Subarrays' is displayed with its difficulty (Easy), accuracy (34.45%), and submission statistics (52K+). The problem description states: 'Given an array of integers `arr[]`, return `true` if it is possible to split it in two subarrays (without reordering the elements), such that the sum of the two subarrays are equal. If it is not possible then return `false`.' Examples provided are:   
 Input: `arr[] = [1, 2, 3, 4, 5, 5]`  
 Output: `true`  
 Explanation: We can divide the array into `[1, 2, 3, 4]` and `[5, 5]`. The sum of both the subarrays are 10.   
 Another example:   
 Input: `arr[] = [4, 3, 2, 1]`  
 Output: `false`  
 Explanation: We cannot divide the array into two subarrays with equal sum.   
 Constraints:   
 $1 \leq \text{arr.size}() \leq 10^5$   
 $1 \leq \text{arr}[i] \leq 10^6$    
 On the right, the Java solution is shown in a code editor. The code is as follows:   

```
1 class Solution {
2     public boolean canSplit(int arr[]) {
3         // code here
4         int lsum=0,rsum=0;
5         for(int i=0;i<arr.length;i++) lsum+=arr[i];
6         for(int i=arr.length-1;i>=0;i--){
7             rsum+=arr[i];
8             lsum-=arr[i];
9             if(lsum==rsum) return true;
10        }
11        return false;
12    }
13 }
```

## 5.1 Minimum Platforms

**Aim :** Given arrival `arr[]` and departure `dep[]` times of trains on the same day, find the minimum number of platforms needed so that no train waits. A platform cannot serve two trains at the same time; if a train arrives before another departs, an extra platform is needed.

**Note:** Time intervals are in the 24-hour format (HHMM) , where the first two characters represent hour (between 00 to 23 ) and the last two characters represent minutes (this will be  $\leq 59$  and  $\geq 0$ ). Leading zeros for hours less than 10 are optional (e.g., 0900 is the same as 900).

```
class Solution {
    public int minPlatform(int arr[], int dep[]) {
        int ans=1;
        Arrays.sort(arr);
        Arrays.sort(dep);
        int i=0,j=0,len=0;
        while(i<arr.length){
            if(arr[i]<=dep[j]){
                len++;
                i++;
            }else{
                len--;
                j++;
            }
            ans=Math.max(len,ans);
        }
        return ans;
    }
}
```

**Output :**

The screenshot shows the GeeksforGeeks website interface for the 'Minimum Platforms' problem. The problem description states: "Given arrival `arr[]` and departure `dep[]` times of trains on the same day, find the minimum number of platforms needed so that no train waits. A platform cannot serve two trains at the same time; if a train arrives before another departs, an extra platform is needed." The note specifies that time intervals are in HHMM format. The solution code is written in Java and is shown in the editor. The code is as follows:

```
1 class Solution {
2     public int minPlatform(int arr[], int dep[]) {
3         // code here
4         int ans=1;
5         Arrays.sort(arr);
6         Arrays.sort(dep);
7         int i=0,j=0,len=0;
8         while(i<arr.length){
9             if(arr[i]<=dep[j]){
10                 len++;
11                 i++;
12             }else{
13                 len--;
14                 j++;
15             }
16             ans=Math.max(len,ans);
17         }
18         return ans;
19     }
20 }
21
```

The 'Compilation Results' section shows 'Compilation Completed' for 'Case 1'. The input is `arr[] =`.

## 5.2 Minimum jumps

**Aim :** You are given an array `arr[]` of non-negative numbers. Each number tells you the maximum number of steps you can jump forward from that position. For example:

- If `arr[i] = 3`, you can jump to index `i + 1`, `i + 2`, or `i + 3` from position `i`.
- If `arr[i] = 0`, you cannot jump forward from that position.

Your task is to find the minimum number of jumps needed to move from the first position in the array to the last position.

**Note:** Return -1 if you can't reach the end of the array.

```
class Solution {
    static int minJumps(int[] arr) {
        if(arr.length<=1) return 0;
        if(arr[0]==0) return -1;
        int jumps=0;
        int n;
        n=arr.length;
        int i;
        i=0;
        while(i<n){
            if(i==n-1){
                return jumps;
            }
            if(arr[i]>0){
                i+=arr[i];
                jumps++;
            }
        }
        return jumps;
    }
}
```

**Output :**

The screenshot displays the GeeksforGeeks IDE interface. On the left, the problem 'Minimum Jumps' is listed with a difficulty of Medium, 11.91% accuracy, 1.1M submissions, and 4 points. The problem description states: 'You are given an array `arr[]` of non-negative numbers. Each number tells you the maximum number of steps you can jump forward from that position. For example: If `arr[i] = 3`, you can jump to index `i + 1`, `i + 2`, or `i + 3` from position `i`. If `arr[i] = 0`, you cannot jump forward from that position. Your task is to find the minimum number of jumps needed to move from the first position in the array to the last position.' Below this, the 'Output Window' shows 'Compilation Results' for 'Case 1' with 'Input: arr[] ='. The main editor on the right shows the Java solution code, which is identical to the one provided in the text block above. The code is for a class `Solution` with a static method `minJumps` that returns the minimum number of jumps required to reach the end of the array.

## 5.3 Minimum Operations

Given a number  $n$ . Find the minimum number of operations required to reach  $n$  starting from 0. You have two operations available:

- Double the number
- Add one to the number

```
class Solution {
    public int minOperation(int n) {
        // code here.
        int ops=0;
        while(n>0){
            if(n%2==0){ n/=2; ops++;}
            else{
                n-=1;
                ops++;
            }
        }
        return ops;
    }
}
```

**Output :**

The screenshot displays the GeeksforGeeks IDE interface. On the left, the problem 'Minimum Operations' is detailed with its difficulty (Easy), accuracy (60.02%), and submission statistics (107K+). The problem description and example are clearly visible. The central code editor shows the Java solution implemented in the `minOperation` method of the `Solution` class. The bottom panel indicates that the code has been successfully compiled and run for the first case, with the input `n = 8` and the expected output `4`.

## 5.4 Stock buy and sell

**Aim :** Given an array `arr[]` denoting the cost of stock on each day, the task is to find the maximum total profit if we can buy and sell the stocks any number of times.

**Note:** We can only sell a stock which we have bought earlier and we cannot hold multiple stocks on any day.

```
class Solution {
    int stockBuySell(int arr[]) {
        // code here
        int sum=0;
        for(int i=1;i<arr.length;i++){
            if(arr[i]>arr[i-1])
                sum+=arr[i]-arr[i-1];
        }
        return sum;
    }
}
```

**Output :**

The screenshot shows the GeeksforGeeks website interface for the 'Stock buy and sell' problem. The problem description states: "Given an array `arr[]` denoting the cost of stock on each day, the task is to find the maximum total profit if we can buy and sell the stocks any number of times. Note: We can only sell a stock which we have bought earlier and we cannot hold multiple stocks on any day." An example is provided: "Input: `arr[] = [100, 180, 260, 310, 40, 535, 695]` Output: 865". The solution code is shown in the editor, and the 'Compilation Results' section indicates 'Compilation Completed' for 'Case 1'.

**Stock buy and sell**

Difficulty: Medium Accuracy: 29.18% Submissions: 305K+ Points: 4 Average Time: 20m

Given an array `arr[]` denoting the cost of stock on each day, the task is to find the maximum total profit if we can buy and sell the stocks any number of times.

**Note:** We can only sell a stock which we have bought earlier and we cannot hold multiple stocks on any day.

**Examples:**

Input: `arr[] = [100, 180, 260, 310, 40, 535, 695]`  
Output: 865

Output Window

Compilation Results Custom Input

Compilation Completed

Case 1

Input: `arr[] =`

Custom Input Compile & Run Submit

## 5.5 Shop in Candy Store

**Aim :** In a candy store, there are different types of candies available and  $\text{prices}[i]$  represent the price of  $i^{\text{th}}$  types of candies. You are now provided with an attractive offer.

For every candy you buy from the store, you can get up to  $k$  other different candies for free. Find the minimum and maximum amount of money needed to buy all the candies.

**Note:** In both cases, you must take the maximum number of free candies possible during each purchase.

```
class Solution {
    public ArrayList<Integer> minMaxCandy(int[] prices, int k) {
        Arrays.sort(prices);
        int n=prices.length;
        int mincost=0,maxcost=0;
        for(int i=0;i<n;i++){
            mincost+=prices[i];
            n-=k;
        }
        int index=-1;
        for(int i=prices.length-1;i>index;i--){
            maxcost+=prices[i];
            index+=k;
        }
        ArrayList<Integer> al=new ArrayList<>();
        al.add(mincost);
        al.add(maxcost);
        return al;
    }
}
```

**Output :**

The screenshot shows the 'Shop in Candy Store' problem on GeeksforGeeks. The problem description states: "In a candy store, there are different types of candies available and  $\text{prices}[i]$  represent the price of  $i^{\text{th}}$  types of candies. You are now provided with an attractive offer. For every candy you buy from the store, you can get up to  $k$  other different candies for free. Find the minimum and maximum amount of money needed to buy all the candies. Note: In both cases, you must take the maximum number of free candies possible during each purchase." Examples are provided: Input:  $\text{prices} = [1, 2, 3, 4, 5]$ ,  $k = 2$ ; Output:  $[[10, 14]]$ . The code editor shows the following Java code:

```
1 class Solution {
2     public ArrayList<Integer> minMaxCandy(int[] prices, int k) {
3         // code here
4         Arrays.sort(prices);
5         int n=prices.length;
6         int mincost=0,maxcost=0;
7         for(int i=0;i<n;i++){
8             mincost+=prices[i];
9             n-=k;
10        }
11        int index=-1;
12        for(int i=prices.length-1;i>index;i--){
13            maxcost+=prices[i];
14            index+=k;
15        }
16        ArrayList<Integer> al=new ArrayList<>();
17        al.add(mincost);
18        al.add(maxcost);
19        return al;
20    }
21 }
22 }
```

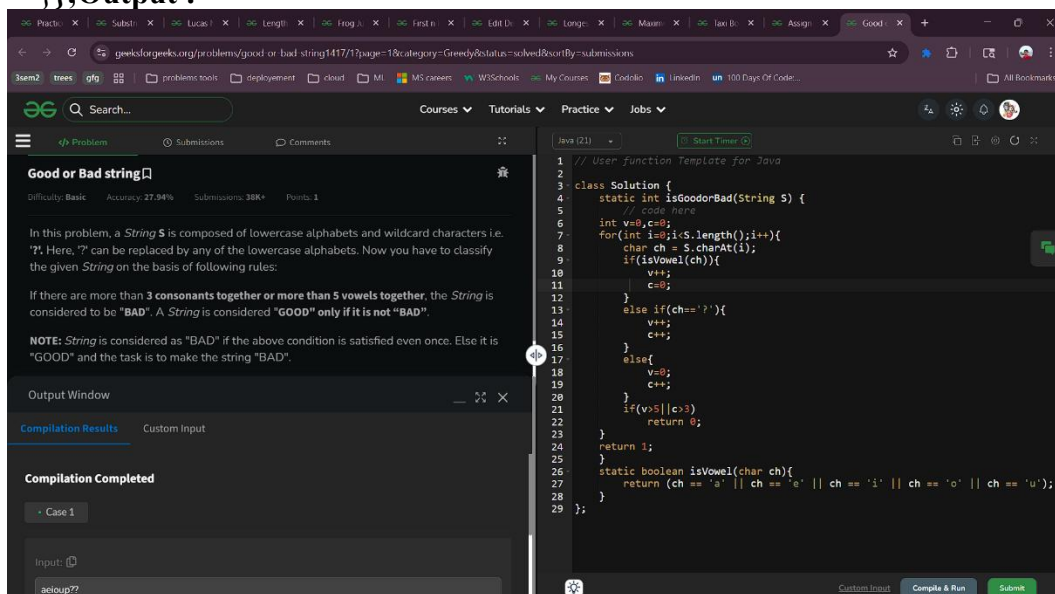
The output window shows "Compilation Completed" and "Case 1". The input field shows "prices[] = [1, 2, 3, 4, 5]" and the output field shows "[10, 14]".

## 5.6 Good or Bad string

**Aim :** In this problem, a *String* *S* is composed of lowercase alphabets and wildcard characters i.e. '?'. Here, '?' can be replaced by any of the lowercase alphabets. Now you have to classify the given *String* on the basis of following rules: If there are more than 3 consonants together or more than 5 vowels together, the *String* is considered to be "BAD". A *String* is considered "GOOD" only if it is not "BAD". NOTE: *String* is considered as "BAD" if the above condition is satisfied even once. Else it is "GOOD" and the task is to make the string "BAD".

```
class Solution {
    static int isGoodorBad(String S) {
        int v=0,c=0;
        for(int i=0;i<S.length();i++){
            char ch = S.charAt(i);
            if(isVowel(ch)){
                v++;
                c=0;
            }
            else if(ch=='?'){
                v++;
                c++;
            }
            else{
                v=0;
                c++;
            }
        }
        if(v>5||c>3)
            return 0;
        return 1;
    }
    static boolean isVowel(char ch){
        return (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u');
    }
};
```

**Output :**



## 5.7 Assign Mice Holes

**Aim :** You are given two arrays `mices[]` and `holes[]` of the same size. The array `mices[]` represents the positions of the mice on a straight line, while the array `holes[]` represents the positions of the holes on the same line. Each hole can accommodate exactly one mouse. A mouse can either stay in its current position, move one step to the right, or move one step to the left, and each move takes one minute. The task is to assign each mouse to a distinct hole in such a way that the time taken by the last mouse to reach its hole is **minimized**.

```
class Solution {
    public int assignHole(int[] mices, int[] holes) {
        // code here
        int max=Integer.MIN_VALUE;
        Arrays.sort(mices);
        Arrays.sort(holes);
        for(int i=0;i<mices.length;i++){
            max=Math.max(Math.abs(mices[i]-holes[i]),max);
        }
        return max;
    }
};
```

**Output :**

The screenshot shows the GeeksforGeeks website interface for the 'Assign Mice Holes' problem. The problem description states: "You are given two arrays `mices[]` and `holes[]` of the same size. The array `mices[]` represents the positions of the mice on a straight line, while the array `holes[]` represents the positions of the holes on the same line. Each hole can accommodate exactly one mouse. A mouse can either stay in its current position, move one step to the right, or move one step to the left, and each move takes one minute. The task is to assign each mouse to a distinct hole in such a way that the time taken by the last mouse to reach its hole is **minimized**." Examples are provided below the description.

The solution code is shown in the editor, which matches the code provided in the text:

```
1 class Solution {
2     public int assignHole(int[] mices, int[] holes) {
3         // code here
4         int max=Integer.MIN_VALUE;
5         Arrays.sort(mices);
6         Arrays.sort(holes);
7         for(int i=0;i<mices.length;i++){
8             max=Math.max(Math.abs(mices[i]-holes[i]),max);
9         }
10        return max;
11    }
12};
```

The 'Compilation Results' section shows 'Compilation Completed' for 'Case 1'. The input is given as `mices[] =`.

## 5.8 Taxi Booking

**Aim :** You are going to book a taxi. There are infinite number of points 1, 2, 3... on the X axis and your current position is cur. There are N Taxis near you, and the position of those taxis is given as an array pos. Where pos[i] denotes the position of the ith taxi. You are also given an array time. Where time[i] denotes the time taken by the ith taxi to cover 1 unit of distance. Your task is to find the minimum time to board a taxi.

```
class Solution {
    public static int minimumTime(int N, int cur, int[] pos, int[] time) {
        // code here
        int min=Integer.MAX_VALUE;
        for(int i=0;i<N;i++){
            int currentTime=Math.abs(pos[i]-cur)*time[i];
            if(currentTime<min){
                min=currentTime;
            }
        }
        return min;
    }
}
```

**Output :**

The screenshot shows the 'Taxi Booking' problem on the GeeksforGeeks website. The problem description states: "You are going to book a taxi. There are infinite number of points 1, 2, 3... on the X axis and your current position is cur. There are N Taxis near you, and the position of those taxis is given as an array pos. Where pos[i] denotes the position of the ith taxi. You are also given an array time. Where time[i] denotes the time taken by the ith taxi to cover 1 unit of distance. Your task is to find the minimum time to board a taxi." Example 1 shows an input of 5 and an output window. The solution code is written in Java and is shown as compiled successfully. The code is as follows:

```
1 class Solution {
2     public static int minimumTime(int N, int cur, int[] pos, int[] time) {
3         // code here
4         int min=Integer.MAX_VALUE;
5         for(int i=0;i<N;i++){
6             int currentTime=Math.abs(pos[i]-cur)*time[i];
7             if(currentTime<min){
8                 min=currentTime;
9             }
10        }
11        return min;
12    }
13 }
14 }
```

The compilation results show "Compilation Completed" for Case 1, with an input of 5 and an output window.

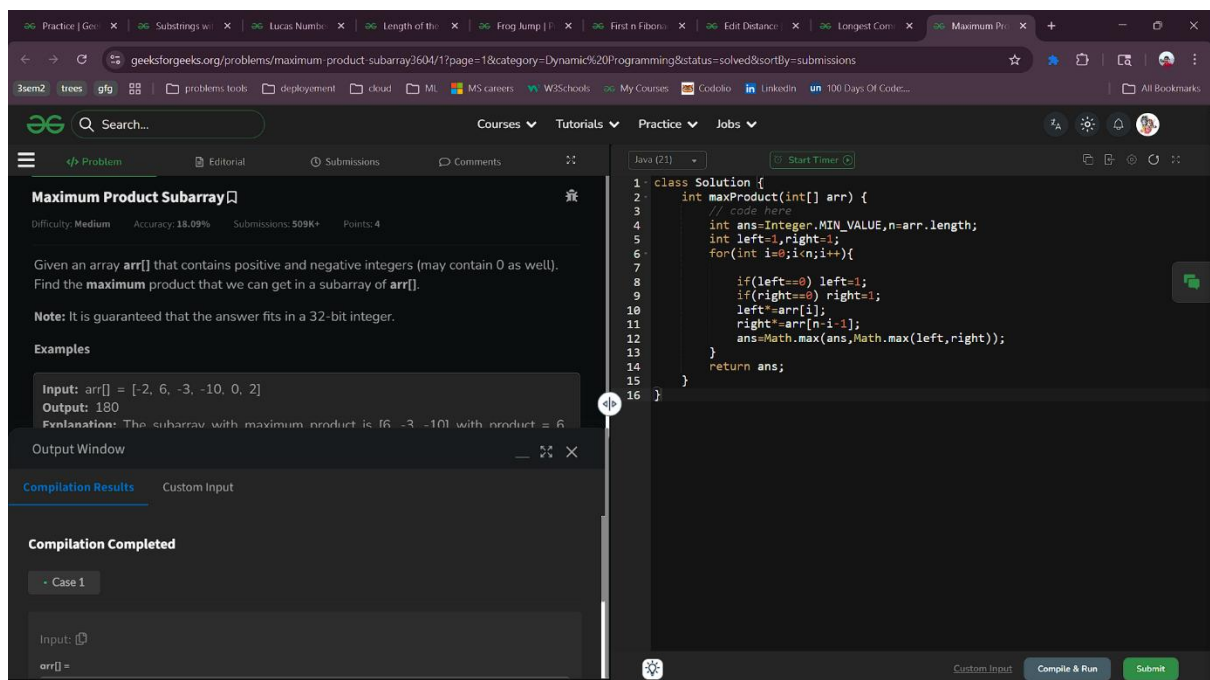
## 6.1 Maximum Product Subarray

**Aim :** Given an array `arr[]` that contains positive and negative integers (may contain 0 as well). Find the maximum product that we can get in a subarray of `arr[]`.

**Note:** It is guaranteed that the answer fits in a 32-bit integer.

```
class Solution {
    int maxProduct(int[] arr) {
        // code here
        int ans=Integer.MIN_VALUE,n=arr.length;
        int left=1,right=1;
        for(int i=0;i<n;i++){
            if(left==0) left=1;
            if(right==0) right=1;
            left*=arr[i];
            right*=arr[n-i-1];
            ans=Math.max(ans,Math.max(left,right));
        }
        return ans;
    }
}
```

**Output :**



## 6.2 Longest Common Subsequence

**Aim :** Given two strings  $s_1$  and  $s_2$ , return the length of their longest common subsequence (LCS). If there is no common subsequence, return 0.

*A subsequence is a sequence that can be derived from the given string by deleting some or no elements without changing the order of the remaining elements. For example, "ABE" is a subsequence of "ABCDE".*

```
class Solution {
    static int lcs(String s1, String s2) {
        // code here
        int m=s1.length(),n=s2.length();
        int dp[][]=new int [m+1][n+1];

        for(int i=1;i<=m;i++){
            for(int j=1;j<=n;j++){
                if(s1.charAt(i-1)==s2.charAt(j-1)){
                    dp[i][j]=1+dp[i-1][j-1];
                }else{
                    dp[i][j]=Math.max(dp[i-1][j],dp[i][j-1]);
                }
            }
        }
        return dp[m][n];
    }
}
```

**Output :**

The screenshot displays the GeeksforGeeks interface for the 'Longest Common Subsequence' problem. The problem description and examples are visible on the left. The code editor on the right contains the following Java code:

```
1 class Solution {
2     static int lcs(String s1, String s2) {
3         // code here
4         int m=s1.length(),n=s2.length();
5         int dp[][]=new int [m+1][n+1];
6
7         for(int i=1;i<=m;i++){
8             for(int j=1;j<=n;j++){
9                 if(s1.charAt(i-1)==s2.charAt(j-1)){
10                     dp[i][j]=1+dp[i-1][j-1];
11                 }else{
12                     dp[i][j]=Math.max(dp[i-1][j],dp[i][j-1]);
13                 }
14             }
15         }
16         return dp[m][n];
17     }
18 }
```

At the bottom left, the 'Compilation Results' section shows 'Compilation Completed' for Case 1. The input field shows 's1 = ABDEFGH'.

### 6.3 Edit Distance

**Aim :** Given two strings *s1* and *s2*. Return the minimum number of operations required to convert *s1* to *s2*.

The possible operations are permitted:

1. Insert a character at any position of the string.
2. Remove any character from the string.
3. Replace any character from the string with any other character.

```
class Solution {
    public int editDistance(String s1, String s2) {
        int m=s1.length();
        int n=s2.length();
        int dp[][]=new int [m+1][n+1];
        for(int i=1;i<=m;i++) dp[i][0]=i;
        for(int j=1;j<=n;j++) dp[0][j]=j;
        for(int i=1;i<=m;i++){
            for(int j=1;j<=n;j++){
                if(s1.charAt(i-1)==s2.charAt(j-1)){
                    dp[i][j]=dp[i-1][j-1];
                }
                else{
                    dp[i][j]=Math.min(dp[i-1][j-1],Math.min(dp[i-1][j],dp[i][j-1]))+1;
                }
            }
        }
        return dp[m][n];
    }
}
```

**Output :**

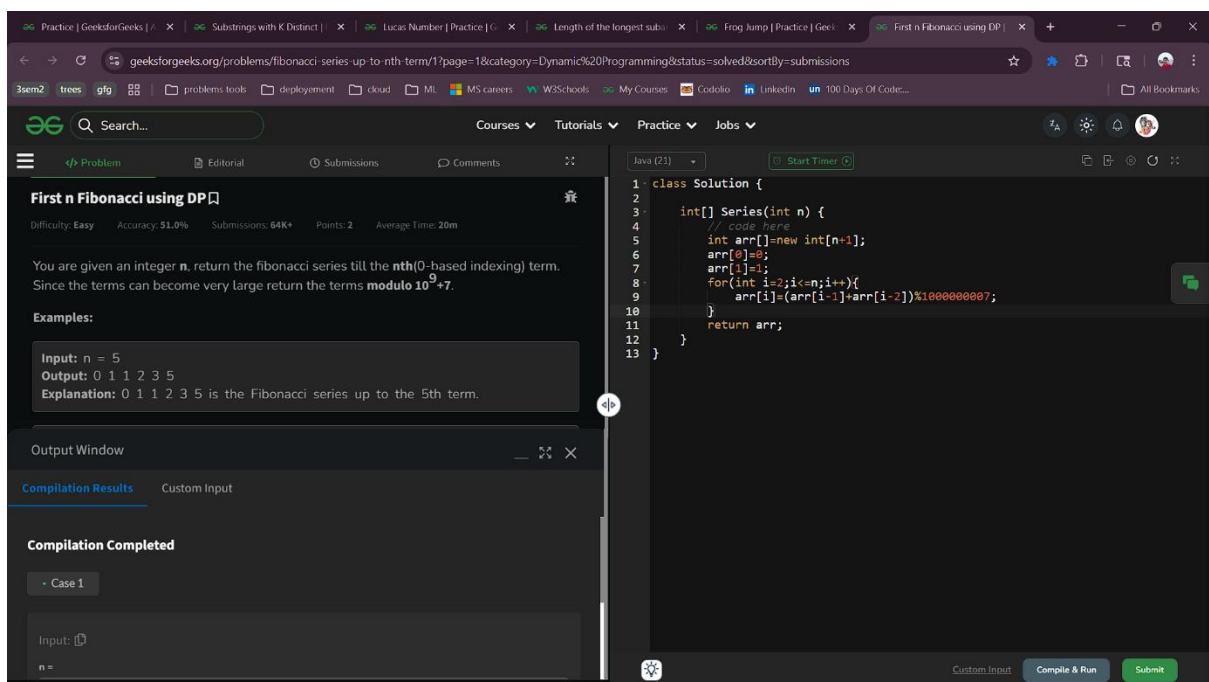
The screenshot displays the 'Edit Distance' problem on the GeeksforGeeks platform. The problem is categorized as 'Hard' with an accuracy of 35.14% and 251K+ submissions. The description states: 'Given two strings *s1* and *s2*. Return the minimum number of operations required to convert *s1* to *s2*. The possible operations are permitted: 1. Insert a character at any position of the string. 2. Remove any character from the string. 3. Replace any character from the string with any other character.' Examples are provided for 's1 = "int", s2 = "insert"' (output 3) and 's1 = "int", s2 = "int"' (output 0). The code editor shows a Java solution using dynamic programming. The 'Compilation Results' section indicates 'Compilation Completed' for Case 1. The input field shows 's1 = '.

## 6.4 First n Fibonacci using DP

**Aim :** You are given an integer  $n$ , return the fibonacci series till the  $n$ th(0-based indexing) term. Since the terms can become very large return the terms modulo  $10^9+7$ .

```
class Solution {
    int[] Series(int n) {
        // code here
        int arr[]=new int[n+1];
        arr[0]=0;
        arr[1]=1;
        for(int i=2;i<=n;i++){
            arr[i]=(arr[i-1]+arr[i-2])%1000000007;
        }
        return arr;
    }
}
```

**Output :**



## 6.5 Frog Jump

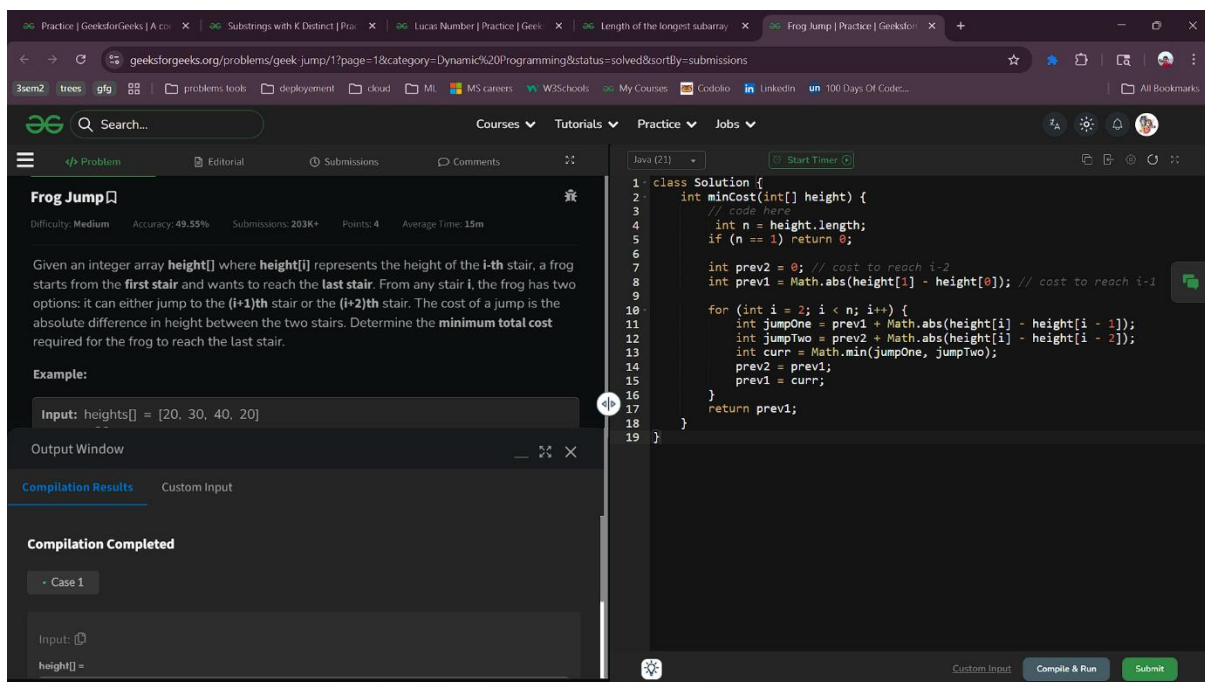
**Aim :** Given an integer array `height[]` where `height[i]` represents the height of the *i*-th stair, a frog starts from the first stair and wants to reach the last stair. From any stair *i*, the frog has two options: it can either jump to the (*i*+1)th stair or the (*i*+2)th stair. The cost of a jump is the absolute difference in height between the two stairs. Determine the minimum total cost required for the frog to reach the last stair.

```
class Solution {
    int minCost(int[] height) {
        // code here
        int n = height.length;
        if (n == 1) return 0;

        int prev2 = 0; // cost to reach i-2
        int prev1 = Math.abs(height[1] - height[0]); // cost to reach i-1

        for (int i = 2; i < n; i++) {
            int jumpOne = prev1 + Math.abs(height[i] - height[i - 1]);
            int jumpTwo = prev2 + Math.abs(height[i] - height[i - 2]);
            int curr = Math.min(jumpOne, jumpTwo);
            prev2 = prev1;
            prev1 = curr;
        }
        return prev1;
    }
}
```

**Output :**



## 6.6 Length of the longest subarray with positive product

**Aim :** Given an array arr[] consisting of n integers, find the length of the longest subarray with positive (non zero) product.

// User function Template for Java

```
class Solution {
    /* Function to return the length of the
       longest subarray with ppositive product */
    int maxLength(int arr[], int n) {
        // code here
        int Pos = 0;
        int Neg = 0;
        int res = 0;

        for (int i = 0; i < n; i++)
        {
            if (arr[i] == 0)
            {
                Pos = Neg = 0;
            }
            else if (arr[i] > 0)
            {
                Pos += 1;

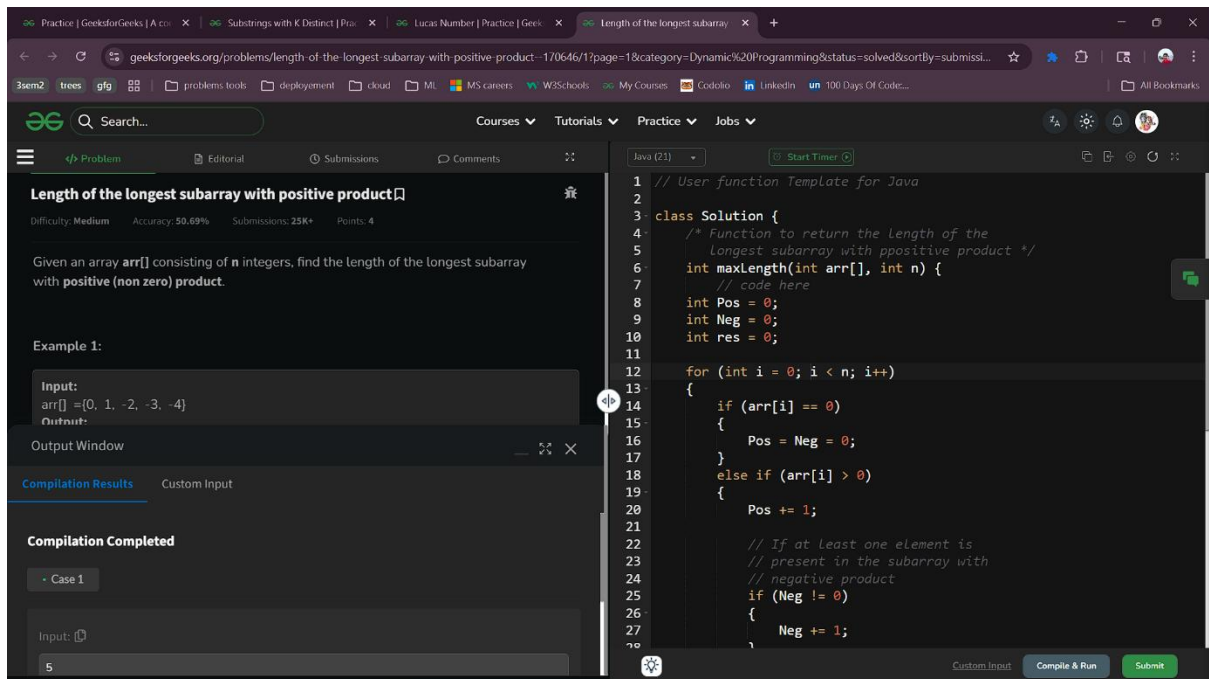
                // If at least one element is
                // present in the subarray with
                // negative product
                if (Neg != 0)
                {
                    Neg += 1;
                }
                res = Math.max(res, Pos);
            }
            else
            {
                Pos = Pos + Neg;
                Neg = Pos - Neg;
                Pos = Pos - Neg;

                // Increment the length of subarray
                // with negative product
                Neg += 1;
                // If at least one element is present
```

```

// in the subarray with positive product
if (Pos != 0)
{
    Pos += 1;
}
res = Math.max(res, Pos);
}
}
return res;
}
}
}Output :

```



## 6.7 Lucas Number

**Aim :** A Lucas Number is a number which is represented by the following recurrence

$$L_n = L_{n-1} + L_{n-2} \text{ for } n > 1$$

$$L_0 = 2$$

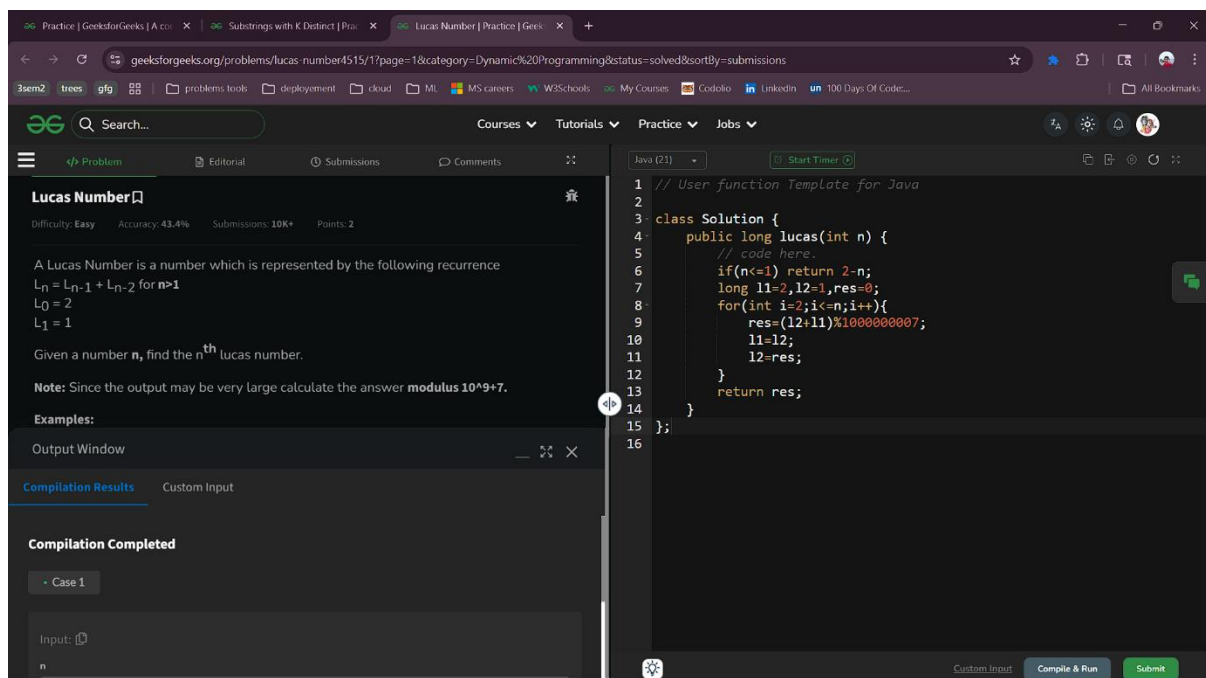
$$L_1 = 1$$

Given a number  $n$ , find the  $n^{\text{th}}$  lucas number.

**Note:** Since the output may be very large calculate the answer modulus  $10^9+7$ .

```
class Solution {
    public long lucas(int n) {
        // code here.
        if(n<=1) return 2-n;
        long l1=2,l2=1,res=0;
        for(int i=2;i<=n;i++){
            res=(l2+l1)%1000000007;
            l1=l2;
            l2=res;
        }
        return res;
    }
};
```

**Output :**

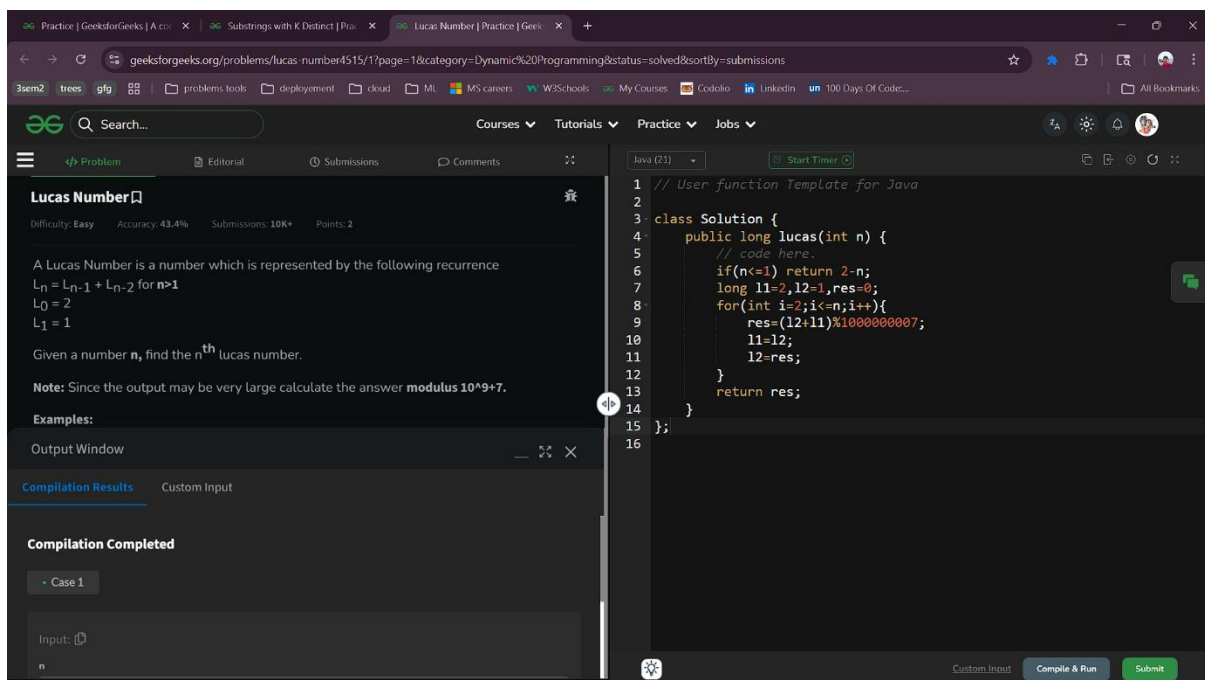


## 6.8 Substrings with K Distinct

**Aim :** You are given a string  $s$  consisting of lowercase characters and an integer  $k$ , You have to count all possible substrings that have exactly  $k$  distinct characters.

```
class Solution {
    static int countSubstr(String s, int k) {
        int n = s.length();
        int ans = 0;
        ans = count(s, k) - count(s, k - 1);
        return ans;
    }
    static int count(String s, int k) {
        int n = s.length();
        int ans = 0;
        int[] freq = new int[26];
        int distinctCnt = 0;
        int i = 0;
        for (int j = 0; j < n; j++) {
            freq[s.charAt(j) - 'a']++;
            if (freq[s.charAt(j) - 'a'] == 1) distinctCnt++;
            while (distinctCnt > k) {
                freq[s.charAt(i) - 'a']--;
                if (freq[s.charAt(i) - 'a'] == 0) distinctCnt--;
                i++;
            }
            ans += j - i + 1;
        }
        return ans;
    }
}
```

**Output :**



## 7.1 Remove Duplicates from Sorted List

**Aim :** Given the head of a sorted linked list, *delete all duplicates such that each element appears only once. Return the linked list sorted as well.*

```
class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        ListNode current = head;

        while (current != null && current.next != null) {
            if (current.val == current.next.val) {
                current.next = current.next.next;
            } else {
                current = current.next;
            }
        }

        return head;
    }
}
```

**Output ;**

The screenshot displays the LeetCode interface for problem 83. The problem description and example 1 are visible on the left. The code editor on the right contains the following Java code:

```
public class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        ListNode current = head;

        while (current != null && current.next != null) {
            if (current.val == current.next.val) {
                current.next = current.next.next;
            } else {
                current = current.next;
            }
        }

        return head;
    }
}
```

The test result at the bottom indicates that the solution is 'Accepted' with a runtime of 0 ms. The input for the test case is head = [1, 1, 2].

## 7.2 Remove Linked List Elements

**Aim :** Given the head of a linked list and an integer `val`, remove all the nodes of the linked list that has `Node.val == val`, and return *the new head*.

```
class Solution {
    public ListNode removeElements(ListNode head, int val) {
        ListNode temp=new ListNode(-1);
        temp.next=head;
        ListNode curr=head,prev=temp;
        while(curr!=null){
            if(curr.val==val){
                prev.next=curr.next;
            }else{
                prev=prev.next;
            }
            curr=curr.next;
        }
        return temp.next;
    }
}
```

**Output :**

The screenshot displays the LeetCode interface for the problem "203. Remove Linked List Elements". The problem description states: "Given the head of a linked list and an integer val, remove all the nodes of the linked list that has Node.val == val, and return the new head." Example 1 shows a linked list [1, 2, 6, 3, 4, 5, 6] being transformed into [1, 2, 3, 4, 5] after removing nodes with value 6. The code editor contains the following Java code:

```
class Solution {
    public ListNode removeElements(ListNode head, int val) {
        ListNode temp=new ListNode(-1);
        temp.next=head;
        ListNode curr=head,prev=temp;
        while(curr!=null){
            if(curr.val==val){
                prev.next=curr.next;
            }else{
                prev=prev.next;
            }
            curr=curr.next;
        }
        return temp.next;
    }
}
```

The test results section shows "Accepted" with a runtime of 0 ms. Three test cases are listed: Case 1, Case 2, and Case 3, all of which are passed.

### 7.3 Reverse Linked List

**Aim :** Given the head of a singly linked list, reverse the list, and return *the reversed list*.

```
class Solution {
    public ListNode reverseList(ListNode head) {
        Stack<ListNode> st=new Stack<>();
        ListNode temp=head;
        while(temp!=null){
            st.push(temp);
            temp=temp.next;
        }
        if(!st.isEmpty()){
            head=st.pop();
            temp=head;
            while(!st.isEmpty()){
                temp.next=st.pop();
                temp=temp.next;
            }
            temp.next=null;
        }
        return head;
    }
}
```

**Output :**

**206. Reverse Linked List**

Given the `head` of a singly linked list, reverse the list, and return *the reversed list*.

**Example 1:**

Input: `head = [1,2,3,4,5]`  
Output: `[5,4,3,2,1]`

**Example 2:**

Input: `head = [1,2]`  
Output: `[2,1]`

```
class Solution {
    public ListNode reverseList(ListNode head) {
        Stack<ListNode> st=new Stack<>();
        ListNode temp=head;
        while(temp!=null){
            st.push(temp);
            temp=temp.next;
        }
        if(!st.isEmpty()){
            head=st.pop();
            temp=head;
            while(!st.isEmpty()){
                temp.next=st.pop();
                temp=temp.next;
            }
            temp.next=null;
        }
        return head;
    }
}
```

**Test Result**

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input: `head = [1,2,3,4,5]`

## 7.4 Palindrome Linked List

**Aim :** Given a matrix `Grid[][]` of size `NxN`. Calculate the absolute difference between the sums of its diagonals.

```
class Solution {
    int diagonalSumDifference(int N, int[][] Grid) {
        // code here
        int diff=0;
        for(int i=0;i<Grid.length;i++){
            diff+=Grid[i][i];
            diff-=Grid[i][Grid.length-i-1];
        }
        return Math.abs(diff);
    }
}
```

**Output :**

The screenshot displays the LeetCode interface for problem 234, "Palindrome Linked List". The problem description asks to return true if a singly linked list is a palindrome. Two examples are provided: Example 1 with a linked list 1 → 2 → 2 → 1 (true) and Example 2 with a linked list 1 → 2 (false). The code editor shows a Java solution using a stack to compare the first and last elements of the list. The test result panel indicates that the solution is "Accepted" with a runtime of 0 ms for two test cases.

## 7.5 Middle of the Linked List

**Aim :** Given the head of a singly linked list, return *the middle node of the linked list*.

**If there are two middle nodes, return the second middle node.**

```
class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode fp=head,sp=head;
        while(fp!=null && fp.next!=null){
            sp=sp.next;
            fp=fp.next.next;
        }
        return sp;
    }
}
```

**Output :**

**876. Middle of the Linked List** Solved

Easy Topics Companies

Given the **head** of a singly linked list, return *the middle node of the linked list*.

If there are two middle nodes, return **the second middle node**.

**Example 1:**

Diagram: 1 → 2 → 3 → 4 → 5 (Node 3 is highlighted in red)

Input: head = [1,2,3,4,5]  
Output: [3,4,5]  
Explanation: The middle node of the list is node 3.

**Example 2:**

Diagram: 1 → 2 → 3 → 4 → 5 → 6 (Node 4 is highlighted in red)

Input: head = [1,2,3,4,5,6]  
Output: [4,5,6]  
Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.

**Code Editor:**

```
class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode fp=head,sp=head;
        while(fp!=null && fp.next!=null){
            sp=sp.next;
            fp=fp.next.next;
        }
        return sp;
    }
}
```

**Test Results:** Accepted Runtime: 0 ms

Case 1 Case 2

Input: head = [1,2,3,4,5]

## 7.6 Remove Nth Node From End of List

**Aim :** Given an nxn matrix .In the given matrix, you have to find the boundary elements of the matrix.

```
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        if(head==null) return null;
        ListNode sp=head,fp=head;
        int i=1;
        while(i++<=n){
            fp=fp.next;
        }
        if(fp == null ) return head.next;
        while(fp.next!=null){
            fp=fp.next;
            sp=sp.next;
        }
        if(sp.next!=null){
            sp.next=sp.next.next;
        }
        return head;
    }
}
```

**Output :**

**19. Remove Nth Node From End of List** Solved

Medium Topics Companies Hint

Given the `head` of a linked list, remove the  $n^{\text{th}}$  node from the end of the list and return its head.

**Example 1:**

Input: `head = [1,2,3,4,5], n = 2`  
 Output: `[1,2,3,5]`

**Example 2:**

Input: `head = [1], n = 1`  
 Output: `[]`

**Code:**

```
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        if(head==null) return null;
        ListNode sp=head,fp=head;
        int i=1;
        while(i++<=n){
            fp=fp.next;
        }
        if(fp == null ) return head.next;
        while(fp.next!=null){
            fp=fp.next;
            sp=sp.next;
        }
        if(sp.next!=null){
            sp.next=sp.next.next;
        }
        return head;
    }
}
```

**Testcase 1 Test Result**

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

head =

[1, 2, 3, 4, 5]

## 7.7 Swap Nodes in Pairs

**Aim :** Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

```
class Solution {
    public ListNode swapPairs(ListNode head) {
        if(head==null || head.next==null) return head;
        ListNode dummy=new ListNode(0);
        dummy.next=head;
        ListNode prev=dummy;
        while(prev.next !=null && prev.next.next != null){
            ListNode first=prev.next;
            ListNode second=first.next;

            first.next=second.next;
            second.next=first;
            prev.next=second;

            prev=first;
        }
        head=dummy.next;
        return head;
    }
}
```

**Output :**

**24. Swap Nodes in Pairs** Solved

Medium Topics Companies

Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

**Example 1:**  
 Input: head = [1, 2, 3, 4]  
 Output: [2, 1, 4, 3]  
 Explanation:

**Example 2:**

13.2K 186 60 Online

**Code**

```
Java
15 dummy.next=head;
16 ListNode prev=dummy;
17 while(prev.next !=null && prev.next.next != null){
18     ListNode first=prev.next;
19     ListNode second=first.next;
20
21     first.next=second.next;
22     second.next=first;
23     prev.next=second;
24
25     prev=first;
26 }
27 head=dummy.next;
28 return head;
29 }
30 }
```

**Test Result**

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3 Case 4

Input: head = [1, 2, 3, 4]

## 7.8 Odd Even Linked List

**Aim :** Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return *the reordered list*. The first node is considered odd, and the second node is even, and so on.

**Note** that the relative order inside both the even and odd groups should remain as it was in the input. You must solve the problem in  $O(1)$  extra space complexity and  $O(n)$  time complexity.

```
class Solution {
    public ListNode oddEvenList(ListNode head) {
        if(head == null || head.next == null) return head;
        ListNode odd = head;
        ListNode even = head.next;
        ListNode evenhead = even;
        while(even != null && even.next != null){
            odd.next = even.next;
            odd = odd.next;
            even.next = even.next.next;
            even = even.next;
        }
        odd.next = evenhead;
        return head;
    }
}
```

**Output :**

The screenshot shows the LeetCode interface for problem 328, "Odd Even Linked List". The problem description states: "Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return the reordered list. The first node is considered odd, and the second node is even, and so on. Note that the relative order inside both the even and odd groups should remain as it was in the input. You must solve the problem in  $O(1)$  extra space complexity and  $O(n)$  time complexity." Example 1 shows an input list [1, 2, 3, 4, 5] being reordered to [1, 3, 5, 2, 4]. Example 2 shows an input list [2, 1, 3, 5, 6, 4, 7] being reordered to [2, 3, 6, 1, 5, 4, 7]. The Java solution code is displayed on the right, implementing the logic described in the text. The test results at the bottom show "Accepted" with a runtime of 0 ms and two test cases passed.

## 8.1 Valid Parentheses

**Aim :** Given a string *s* containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

**An input string is valid if:**

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

```
class Solution {
    public boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();
        for (char ch : s.toCharArray()) {
            if (ch == '(' || ch == '[' || ch == '{') {
                stack.push(ch);
            } else {
                if (stack.isEmpty()) {
                    return false;
                }
                char top = stack.pop();
                if (ch == ')' && top != '(') {
                    return false;
                }
                if (ch == ']' && top != '[') {
                    return false;
                }
                if (ch == '}' && top != '{') {
                    return false;
                }
            }
        }
        return stack.isEmpty();
    }
}
```

**Output :**

The screenshot shows the LeetCode interface for problem 20, 'Valid Parentheses'. The left sidebar contains the problem description, including the aim and the three conditions for a valid string. The main area on the right shows a Java code editor with the solution code. Below the code editor, there is a 'Test Result' section with the message 'You must run your code first.'

## 8.2 Implement Queue using Stacks

**Aim :** Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

**Implement the MyQueue class:**

- **void push(int x)** Pushes element x to the back of the queue.
- **int pop()** Removes the element from the front of the queue and returns it.
- **int peek()** Returns the element at the front of the queue.
- **boolean empty()** Returns true if the queue is empty, false otherwise.

```
class MyQueue {
    Stack<Integer> s1;
    Stack<Integer> s2;
    public MyQueue() {
        s1=new Stack<>();
        s2=new Stack<>();
    }
    private int front;
    public void push(int x) {
        if(s1.isEmpty()) front=x;
        s1.push(x);
    }

    public int pop() {
        if(s2.isEmpty()){
            while(!s1.isEmpty()) s2.push(s1.pop());
        }
        return s2.pop();
    }
}
```

```
public int peek() {
    if(!s2.isEmpty()) return s2.peek();
    return front;
}

public boolean empty() {
    return s1.isEmpty() &&
    s2.isEmpty();
}
```

**Output ;**

**232. Implement Queue using Stacks** Solved

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

Implement the `MyQueue` class:

- `void push(int x)` Pushes element x to the back of the queue.
- `int pop()` Removes the element from the front of the queue and returns it.
- `int peek()` Returns the element at the front of the queue.
- `boolean empty()` Returns true if the queue is empty, false otherwise.

**Notes:**

- You must use **only** standard operations of a stack, which means **only** push to top, peek/pop from top, size, and is empty operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

**Example 1:**

**Input:** ["MyQueue", "push", "push", "peek", "pop", "empty"]

**Output:** [null, null, null, 2, 1, true]

```
class MyQueue {
    Stack<Integer> s1;
    Stack<Integer> s2;
    public MyQueue() {
        s1=new Stack<>();
        s2=new Stack<>();
    }
    private int front;
    public void push(int x) {
        if(s1.isEmpty()) front=x;
        s1.push(x);
    }
    public int pop() {
        if(s2.isEmpty()){
            while(!s1.isEmpty()) s2.push(s1.pop());
        }
        return s2.pop();
    }
    public int peek() {
        if(!s2.isEmpty()) return s2.peek();
        return front;
    }
    public boolean empty() {
        return s1.isEmpty() && s2.isEmpty();
    }
}
```

Accepted Runtime: 0 ms

Case 1

Input: ["MyQueue", "push", "push", "peek", "pop", "empty"]

### 8.3 Next Greater Element I

**Aim :** The next greater element of some element  $x$  in an array is the first greater element that is to the right of  $x$  in the same array.

You are given two distinct 0-indexed integer arrays `nums1` and `nums2`, where `nums1` is a subset of `nums2`.

For each  $0 \leq i < \text{nums1.length}$ , find the index  $j$  such that `nums1[i] == nums2[j]` and determine the next greater element of `nums2[j]` in `nums2`. If there is no next greater element, then the answer for this query is `-1`. Return an array `ans` of length `nums1.length` such that `ans[i]` is the next greater element as described above.

```
class Solution {
    public int[] nextGreaterElement(int[] nums1, int[] nums2) {
        int[] nextGreater = new int[10001];
        Stack<Integer> stack = new Stack<>();

        for (int i = nums2.length - 1; i >= 0; i--) {
            while (!stack.isEmpty() && stack.peek() <= nums2[i]) {
                stack.pop();
            }
            nextGreater[nums2[i]] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(nums2[i]);
        }

        for (int i = 0; i < nums1.length; i++) {
            nums1[i] = nextGreater[nums1[i]];
        }

        return nums1;
    }
}
```

**Output :**

The screenshot shows the LeetCode interface for problem 496. The problem description states: "The next greater element of some element  $x$  in an array is the first greater element that is to the right of  $x$  in the same array. You are given two distinct 0-indexed integer arrays `nums1` and `nums2`, where `nums1` is a subset of `nums2`. For each  $0 \leq i < \text{nums1.length}$ , find the index  $j$  such that `nums1[i] == nums2[j]` and determine the next greater element of `nums2[j]` in `nums2`. If there is no next greater element, then the answer for this query is `-1`. Return an array `ans` of length `nums1.length` such that `ans[i]` is the next greater element as described above."

The Java solution provided in the code editor is:

```
class Solution {
    public int[] nextGreaterElement(int[] nums1, int[] nums2) {
        int[] nextGreater = new int[10001];
        Stack<Integer> stack = new Stack<>();

        for (int i = nums2.length - 1; i >= 0; i--) {
            while (!stack.isEmpty() && stack.peek() <= nums2[i]) {
                stack.pop();
            }
            nextGreater[nums2[i]] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(nums2[i]);
        }

        for (int i = 0; i < nums1.length; i++) {
            nums1[i] = nextGreater[nums1[i]];
        }
    }
}
```

The test results show that the solution is "Accepted" with a runtime of 0 ms. The input is `nums1 = [4,1,2], nums2 = [1,3,4,2]` and the output is `[-1,3,-1]`. The explanation states: "The next greater element for each value of `nums1` is as follows: - 4 is underlined in `nums2 = [1,3,4,2]`. There is no next greater element, so the answer is `-1`. - 1 is underlined in `nums2 = [1,3,4,2]`. The next greater element is 3. - 2 is underlined in `nums2 = [1,3,4,2]`. There is no next greater element, so the answer is `-1`."



## 8.5 Remove All Adjacent Duplicates In String

**Aim :** You are given a string *s* consisting of lowercase English letters. A duplicate removal consists of choosing two adjacent and equal letters and removing them.

We repeatedly make duplicate removals on *s* until we no longer can.

Return *the final string after all such duplicate removals have been made*. It can be proven that the answer is unique.

```
class Solution {
    public String removeDuplicates(String s) {
        Stack<Character> st=new Stack<>();
        for(int i=s.length()-1;i>=0;i--){
            if(st.empty()) st.push(s.charAt(i));
            else if(st.peek()==s.charAt(i)){
                //st.push(s.charAt(i));
                st.pop();
            }
            else st.push(s.charAt(i));
        }
        StringBuilder sb=new StringBuilder();
        while(!st.empty()){
            sb.append(st.pop());
        }
        return sb.toString();
    }
}
```

**Output :**

The screenshot shows the LeetCode interface for problem 1047. The description states: "You are given a string *s* consisting of lowercase English letters. A **duplicate removal** consists of choosing two **adjacent** and **equal** letters and removing them. We repeatedly make **duplicate removals** on *s* until we no longer can. Return the **final string after all such duplicate removals have been made**. It can be proven that the answer is **unique**."

**Example 1:**  
Input: *s* = "abbaca"  
Output: "ca"  
Explanation: For example, in "abbaca" we could remove "bb" since the letters are adjacent and equal, and this is the only possible move. The result of this move is that the string is "aaca", of which only "aa" is possible, so the final string is "ca".

**Example 2:**  
Input: *s* = "azxxzy"  
Output: "ay"

The code editor shows the following Java solution:

```
1 class Solution {
2     public String removeDuplicates(String s) {
3         Stack<Character> st=new Stack<>();
4         for(int i=s.length()-1;i>=0;i--){
5             if(st.empty()) st.push(s.charAt(i));
6             else if(st.peek()==s.charAt(i)){
7                 //st.push(s.charAt(i));
8                 st.pop();
9             }
10            else st.push(s.charAt(i));
11        }
12        StringBuilder sb=new StringBuilder();
13        while(!st.empty()){
14            sb.append(st.pop());
15        }
16        return sb.toString();
17    }
18 }
```

The Test Result section shows "Accepted" with a runtime of 0 ms. Two test cases are passed:

- Case 1: Input: *s* = "abbaca", Output: "ca"
- Case 2: Input: *s* = "azxxzy", Output: "ay"

## 9.1 Implement Stack using Queues

**Aim :** Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (push, top, pop, and empty).

**Implement the MyStack class:**

- **void push(int x)** Pushes element *x* to the top of the stack.
- **int pop()** Removes the element on the top of the stack and returns it.
- **int top()** Returns the element on the top of the stack.
- **boolean empty()** Returns true if the stack is empty, false otherwise.

```
class MyStack {
    Queue<Integer> q;
    public MyStack() {
        q=new LinkedList<>();
    }

    public void push(int x) {
        q.offer(x);
        for(int i=1;i<q.size();i++) q.offer(q.poll());
    }

    public int pop() {
        if(q.isEmpty()) return -1;
        return q.poll();
    }
}
```

```
public int top() {
    if(q.isEmpty()) return -1;
    return q.peek();
}

public boolean empty() {
    return q.isEmpty();
}
```

**Output :**

The screenshot shows the LeetCode interface for the problem "225. Implement Stack using Queues". The problem description states: "Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (push, top, pop, and empty). Implement the MyStack class: void push(int x) Pushes element x to the top of the stack. int pop() Removes the element on the top of the stack and returns it. int top() Returns the element on the top of the stack. boolean empty() Returns true if the stack is empty, false otherwise. Notes: You must use only standard operations of a queue, which means that only push to back, peek/pop from front, size and is empty operations are valid. Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's standard operations. Example 1: Input: ["MyStack", "push", "push", "top", "pop", "empty"]". The solution code is written in Java and uses two queues. The test result shows "Accepted" with a runtime of 0 ms.

## 9.2 Sliding Window Maximum

**Aim :** You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position. Return *the max sliding window*.

```
class Solution {
    public int[] maxSlidingWindow(int[] a, int k) {
        int n=a.length;
        int res[]=new int[n-k+1];
        Deque<Integer> dq=new LinkedList<>();
        for(int i=0;i<k;i++){
            while(!dq.isEmpty() && a[dq.peekLast()]<=a[i])
                dq.removeLast();
            dq.addLast(i);
        }
        res[0]=a[dq.peekFirst()];
        for(int i=k;i<n;i++){
            while(!dq.isEmpty() && dq.peekFirst()<=i-k)
                dq.removeFirst();
            while(!dq.isEmpty() && a[dq.peekLast()]<=a[i])
                dq.removeLast();
            dq.addLast(i);
            res[i-k+1]=a[dq.peekFirst()];
        }
        return res;
    }
}
```

**Output ;**

The screenshot shows the LeetCode interface for problem 239. The problem description states: "You are given an array of integers `nums`, there is a sliding window of size `k`, which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position. Return the max sliding window."

**Example 1:**  
Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`  
Output: `[3,3,5,5,6,7]`  
Explanation: The window moves from left to right, and the maximum value in each window is recorded.

The code editor shows the following Java solution:

```
class Solution {
    public int[] maxSlidingWindow(int[] a, int k) {
        int n=a.length;
        int res[]=new int[n-k+1];
        Deque<Integer> dq=new LinkedList<>();
        for(int i=0;i<k;i++){
            while(!dq.isEmpty() && a[dq.peekLast()]<=a[i])
                dq.removeLast();
            dq.addLast(i);
        }
        res[0]=a[dq.peekFirst()];
        for(int i=k;i<n;i++){
            while(!dq.isEmpty() && dq.peekFirst()<=i-k)
                dq.removeFirst();
            while(!dq.isEmpty() && a[dq.peekLast()]<=a[i])
                dq.removeLast();
            dq.addLast(i);
            res[i-k+1]=a[dq.peekFirst()];
        }
        return res;
    }
}
```

The test result section shows "Accepted" with a runtime of 0 ms. Below the test result, the input and output for the test case are displayed:

Input: `nums = [1, 3, -1, -3, 5, 3, 6, 7]`  
Output: `[3, 3, 5, 5, 6, 7]`

### 9.3 Kth Largest Element in an Array

**Aim :** Given an integer array `nums` and an integer `k`, return *the  $k^{\text{th}}$  largest element in the array*.

Note that it is the  $k^{\text{th}}$  largest element in the sorted order, not the  $k^{\text{th}}$  distinct element.

Can you solve it without sorting?

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        PriorityQueue<Integer> pq=new PriorityQueue<>();
        for(int i:nums){
            pq.add(i);
            if(pq.size()>k)
                pq.poll();
        }
        return pq.peek();
    }
}
```

**Output :**

The screenshot displays the LeetCode interface for problem 215. The left panel contains the problem description: "215. Kth Largest Element in an Array" (Medium difficulty). It states the goal is to return the  $k^{\text{th}}$  largest element in the array. Example 1 shows input `nums = [3,2,1,5,6,4]` and `k = 2` resulting in output `5`. Example 2 shows input `nums = [3,2,3,1,2,4,5,6]` and `k = 4` resulting in output `4`. Constraints specify  $1 \leq k \leq \text{nums.length} \leq 10^5$  and  $-10^4 \leq \text{nums}[i] \leq 10^4$ . The right panel shows a Java solution using a `PriorityQueue` to maintain the  $k$  largest elements. The code is as follows:

```
1 class Solution {
2     public int findKthLargest(int[] nums, int k) {
3         PriorityQueue<Integer> pq=new PriorityQueue<>();
4         for(int i:nums){
5             pq.add(i);
6             if(pq.size()>k)
7                 pq.poll();
8         }
9         return pq.peek();
10    }
11 }
```

Below the code, the 'Test Result' section shows 'Accepted' with a runtime of 0 ms. Two test cases are passed: Case 1 and Case 2. The input for Case 1 is `nums = [3,2,1,5,6,4]`.

## 9.4 Sort Characters By Frequency

**Aim :** Given a string *s*, sort it in decreasing order based on the frequency of the characters. The frequency of a character is the number of times it appears in the string.

**Return** *the sorted string*. If there are multiple answers, return *any of them*.

```
class Solution {
    public String frequencySort(String s) {
        TreeMap<Character,Integer> tm=new TreeMap<>();
        for(char ch:s.toCharArray()){
            tm.put(ch,tm.getOrDefault(ch,0)+1);
        }
        PriorityQueue<Map.Entry<Character, Integer>> pq = new PriorityQueue<>(
            (a, b) -> b.getValue() - a.getValue()
        );

        pq.addAll(tm.entrySet());

        StringBuilder result = new StringBuilder();
        while (!pq.isEmpty()) {
            Map.Entry<Character, Integer> entry = pq.poll();
            result.append(String.valueOf(entry.getKey()).repeat(entry.getValue()));
        }

        return result.toString();
    }
}
```

**Output :**

The screenshot shows the LeetCode interface for problem 451. The problem description states: "Given a string *s*, sort it in decreasing order based on the frequency of the characters. The frequency of a character is the number of times it appears in the string. Return the sorted string. If there are multiple answers, return any of them."

**Example 1:**  
Input: *s* = "tree"  
Output: "eert"  
Explanation: 'e' appears twice while 'r' and 't' both appear once. So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.

**Example 2:**  
Input: *s* = "cccaaa"  
Output: "aaaccc"  
Explanation: Both 'c' and 'a' appear three times, so both "cccaaa" and "aaaccc" are valid answers. Note that "cacaca" is incorrect, as the same characters must be together.

The code editor shows the following Java solution:

```
class Solution {
    public String frequencySort(String s) {
        TreeMap<Character,Integer> tm=new TreeMap<>();
        for(char ch:s.toCharArray()){
            tm.put(ch,tm.getOrDefault(ch,0)+1);
        }
        PriorityQueue<Map.Entry<Character, Integer>> pq = new PriorityQueue<>(
            (a, b) -> b.getValue() - a.getValue()
        );

        pq.addAll(tm.entrySet());

        StringBuilder result = new StringBuilder();
        while (!pq.isEmpty()) {
            Map.Entry<Character, Integer> entry = pq.poll();
            result.append(String.valueOf(entry.getKey()).repeat(entry.getValue()));
        }

        return result.toString();
    }
}
```

The test results section shows "Accepted" with a runtime of 1 ms. Three test cases are listed, all of which are passed (Case 1, Case 2, Case 3).

## 9.5 Minimum Operations to Make Binary Array Elements Equal to One I

**Aim :** You can do the following operation on the array any number of times (possibly zero):

- Choose any 3 consecutive elements from the array and flip all of them.

Flipping an element means changing its value from 0 to 1, and from 1 to 0.

Return the minimum number of operations required to make all elements in `nums` equal to 1. If it is impossible, return -1.

```
class Solution {
    public int minOperations(int[] nums) {
        int cnt=0;
        for(int i=2;i<nums.length;i++){
            if(nums[i-2]==0){
                cnt++;
                nums[i-2]=nums[i-2]^1;
                nums[i-1]=nums[i-1]^1;
                nums[i]=nums[i]^1;
            }
        }
        int sum=0;
        for(int i:nums) sum+=i;
        if(sum==nums.length) return cnt;
        return -1;
    }
}
```

**Output :**

The screenshot shows the LeetCode interface for problem 3191. The problem description states: "You are given a binary array `nums`. You can do the following operation on the array any number of times (possibly zero): Choose any 3 consecutive elements from the array and flip all of them. Flipping an element means changing its value from 0 to 1, and from 1 to 0. Return the minimum number of operations required to make all elements in `nums` equal to 1. If it is impossible, return -1." An example is provided: Input: `nums = [0,1,1,1,0,0]`, Output: 3. The explanation shows two operations: flipping indices 0,1,2 and then indices 1,2,3. The Java solution in the editor is identical to the one provided in the text. The test results show 'Accepted' with a runtime of 0 ms for two cases.

## 10.1 Insert into a Binary Search Tree

**Aim :** You are given the root node of a binary search tree (BST) and a value to insert into the tree. Return *the root node of the BST after the insertion*. It is guaranteed that the new value does not exist in the original BST.

Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return any of them.

```
class Solution {
    public TreeNode insertIntoBST(TreeNode root, int val) {
        if(root == null) return new TreeNode(val);
        TreeNode curr = root;
        while(true){
            if(curr.val < val){
                if(curr.right != null) curr = curr.right;
                else {
                    curr.right = new TreeNode(val);
                    break;
                }
            }
            else{
                if(curr.left != null) curr = curr.left;
                else{
                    curr.left = new TreeNode(val);
                    break;
                }
            }
        }
        return root;
    }
}
```

**Output :**

The screenshot displays the LeetCode interface for the problem "701. Insert into a Binary Search Tree". On the left, the problem description states: "You are given the root node of a binary search tree (BST) and a value to insert into the tree. Return the root node of the BST after the insertion. It is guaranteed that the new value does not exist in the original BST. Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return any of them." An example shows a tree with root 4, left child 2, and right child 7. Node 2 has children 1 and 3. The input is root = [4,2,7,1,3], val = 5, and the output is [4,2,7,1,3,5].

The code editor on the right contains the following Java code:

```
class Solution {
    public TreeNode insertIntoBST(TreeNode root, int val) {
        if (root == null) return new TreeNode(val);
        TreeNode curr = root;
        while (true) {
            if (curr.val < val) {
                if (curr.right != null) curr = curr.right;
                else {
                    curr.right = new TreeNode(val);
                    break;
                }
            } else {
                if (curr.left != null) curr = curr.left;
                else {
                    curr.left = new TreeNode(val);
                    break;
                }
            }
        }
        return root;
    }
}
```

The test result at the bottom shows "Accepted" with a runtime of 0 ms. Three test cases (Case 1, Case 2, Case 3) are all passed.

## 10.2 Kth Smallest Element in a BST

**Aim :** Given the root of a binary search tree, and an integer  $k$ , return *the  $k^{\text{th}}$  smallest value (1-indexed) of all the values of the nodes in the tree.*

```
class Solution {
    int count = 0;
    TreeNode result = null;

    public int kthSmallest(TreeNode root, int k) {
        helper(root, k);
        return result.val;
    }

    private void helper(TreeNode node, int k) {
        if (node == null) return;

        helper(node.left, k);    // Traverse left
        count++;                // Visit current
        if (count == k) {
            result = node;
            return;
        }
        helper(node.right, k);   // Traverse right
    }
}
```

**Output ;**

The screenshot displays the LeetCode interface for problem 230. The problem description and example are visible on the left. The code editor on the right contains the following Java code:

```
class Solution {
    int count = 0;
    TreeNode result = null;

    public int kthSmallest(TreeNode root, int k) {
        helper(root, k);
        return result.val;
    }

    private void helper(TreeNode node, int k) {
        if (node == null) return;

        helper(node.left, k);
        count++;
        if (count == k) {
            result = node;
            return;
        }
        helper(node.right, k);
    }
}
```

The test result section shows 'Accepted' with a runtime of 0 ms. The input and output for the test case are also displayed.

### 10.3 Range Sum of BST

**Aim :** Given the root node of a binary search tree and two integers low and high, return *the sum of values of all nodes with a value in the inclusive range [low, high]*.

```
class Solution {
    public int rangeSumBST(TreeNode root, int low, int high) {
        if(root == null) return 0;
        int currentval=(root.val >= low && root.val <=high)?root.val:0;
        return currentval + rangeSumBST(root.left,low,high) +
        rangeSumBST(root.right,low,high);
    }
}
```

**Output :**

The screenshot shows a web browser with the LeetCode problem "938. Range Sum of BST" open. The problem description states: "Given the root node of a binary search tree and two integers low and high, return the sum of values of all nodes with a value in the inclusive range [low, high]." An example 1 shows a binary search tree with root 10, left child 5, right child 15, and further children 3, 7, and 18. The input is root = [10,5,15,3,7,null,18], low = 7, high = 15, and the output is 32. The explanation states: "Nodes 7, 10, and 15 are in the range [7, 15]. 7 + 10 + 15 = 32." Below the problem description, there is a code editor with a Java solution. The code is as follows:

```
10     this.val = val;
11     this.left = left;
12     this.right = right;
13 }
14 }
15 }
16 class Solution {
17     public int rangeSumBST(TreeNode root, int low, int high) {
18         if(root == null) return 0;
19         int currentval=(root.val >= low && root.val <=high)?root.val:0;
20         return currentval + rangeSumBST(root.left,low,high) + rangeSumBST(root.right,low,
21         high);
22     }
23 }
```

The code editor also shows a "Test Result" section with "Accepted" status and "Runtime: 0 ms". There are two test cases, Case 1 and Case 2, both of which are passed.

## 10.4 Search in a Binary Search Tree

**Aim :** You are given the root of a binary search tree (BST) and an integer val.

**Find the node in the BST that the node's value equals val and return the subtree rooted with that node. If such a node does not exist, return null.**

```
class Solution {
    public TreeNode searchBST(TreeNode root, int val) {
        if(root==null) return null;
        while(root!=null && root.val!=val){
            if(root.val<val) root=root.right;
            else root=root.left;
        }
        return root;
    }
}
```

**Output :**

**700. Search in a Binary Search Tree** Solved

Easy Topics Companies

You are given the `root` of a binary search tree (BST) and an integer `val`.

Find the node in the BST that the node's value equals `val` and return the subtree rooted with that node. If such a node does not exist, return `null`.

**Example 1:**

**Input:** `root = [4,2,7,1,3]`, `val = 2`

**Code:**

```
class Solution {
    public TreeNode searchBST(TreeNode root, int val) {
        if(root==null) return null;
        while(root!=null && root.val!=val){
            if(root.val<val) root=root.right;
            else root=root.left;
        }
        return root;
    }
}
```

**Test Result:** Accepted Runtime: 0 ms

Case 1 Case 2

Input: root = [4,2,7,1,3], val = 2

## 10.5 Two Sum IV - Input is a BST

**Aim :** Given the root of a binary search tree and an integer  $k$ , return true *if there exist two elements in the BST such that their sum is equal to  $k$ , or false otherwise.*

```
class Solution {
    public boolean findTarget(TreeNode root, int target) {
        ArrayList<Integer> al=new ArrayList<>();
        getInorder(root,al);
        int s=0;
        int e=al.size()-1;
        while(s<e){
            if(al.get(s)+al.get(e)==target) return true;
            else if(al.get(s)+al.get(e)< target) s++;
            else e--;
        }
        return false;
    }

    private static void getInorder(TreeNode root,ArrayList<Integer> al){
        if(root == null ) return;
        getInorder(root.left,al);
        al.add(root.val);
        getInorder(root.right,al);
    }
}
```

**Output :**

The screenshot shows a web browser with the LeetCode problem page for "653. Two Sum IV - Input is a BST". The problem description states: "Given the root of a binary search tree and an integer k, return true if there exist two elements in the BST such that their sum is equal to k, or false otherwise." Example 1 shows a BST with root 5, left child 3, right child 6, and further children 2, 4, 7. The input is root = [5,3,6,2,4,null,7], k = 9, and the output is true. The code editor shows the following Java code:

```
class Solution {
    public boolean findTarget(TreeNode root, int target) {
        ArrayList<Integer> al=new ArrayList<>();
        getInorder(root,al);
        int s=0;
        int e=al.size()-1;
        while(s<e){
            if(al.get(s)+al.get(e)==target) return true;
            else if(al.get(s)+al.get(e)< target) s++;
            else e--;
        }
        return false;
    }

    private static void getInorder(TreeNode root,ArrayList<Integer> al){
        if(root == null ) return;
        getInorder(root.left,al);
        al.add(root.val);
        getInorder(root.right,al);
    }
}
```

The test result shows "Accepted" with a runtime of 0 ms. Two test cases are listed: Case 1 and Case 2, both of which are passed.

## 10.6 Minimum Absolute Difference in BST

**Aim :** Given the root of a Binary Search Tree (BST), return *the minimum absolute difference between the values of any two different nodes in the tree.*

```
class Solution {
    public int getMinimumDifference(TreeNode root) {
        ArrayList<Integer> al=new ArrayList<>();
        inorder(root,al);
        int min=Integer.MAX_VALUE;
        for(int i=1;i<al.size();i++){
            min=Math.min(min,Math.abs(al.get(i-1)-al.get(i)));
        }
        return min;
    }
    static void inorder(TreeNode root,ArrayList<Integer> al){
        if(root == null ) return ;
        inorder(root.left,al);
        al.add(root.val);
        inorder(root.right,al);
    }
}
```

**Output :**

The screenshot displays the LeetCode interface for the problem "530. Minimum Absolute Difference in BST". On the left, the problem description states: "Given the root of a Binary Search Tree (BST), return the minimum absolute difference between the values of any two different nodes in the tree." Below this, an example tree is shown with root 4, left child 2, right child 6, and 2 having children 1 and 3. The input is given as "root = [4,2,6,1,3]" and the output is "1".

The code editor on the right contains the following Java code:

```
class Solution {
    public int getMinimumDifference(TreeNode root) {
        ArrayList<Integer> al=new ArrayList<>();
        inorder(root,al);
        int min=Integer.MAX_VALUE;
        for(int i=1;i<al.size();i++){
            min=Math.min(min,Math.abs(al.get(i-1)-al.get(i)));
        }
        return min;
    }
    static void inorder(TreeNode root,ArrayList<Integer> al){
        if(root == null ) return ;
        inorder(root.left,al);
        al.add(root.val);
        inorder(root.right,al);
    }
}
```

The test result at the bottom shows "Accepted" with a runtime of 0 ms. Two test cases are listed: Case 1 and Case 2, both of which passed.

## 10.7 Ceil in BST

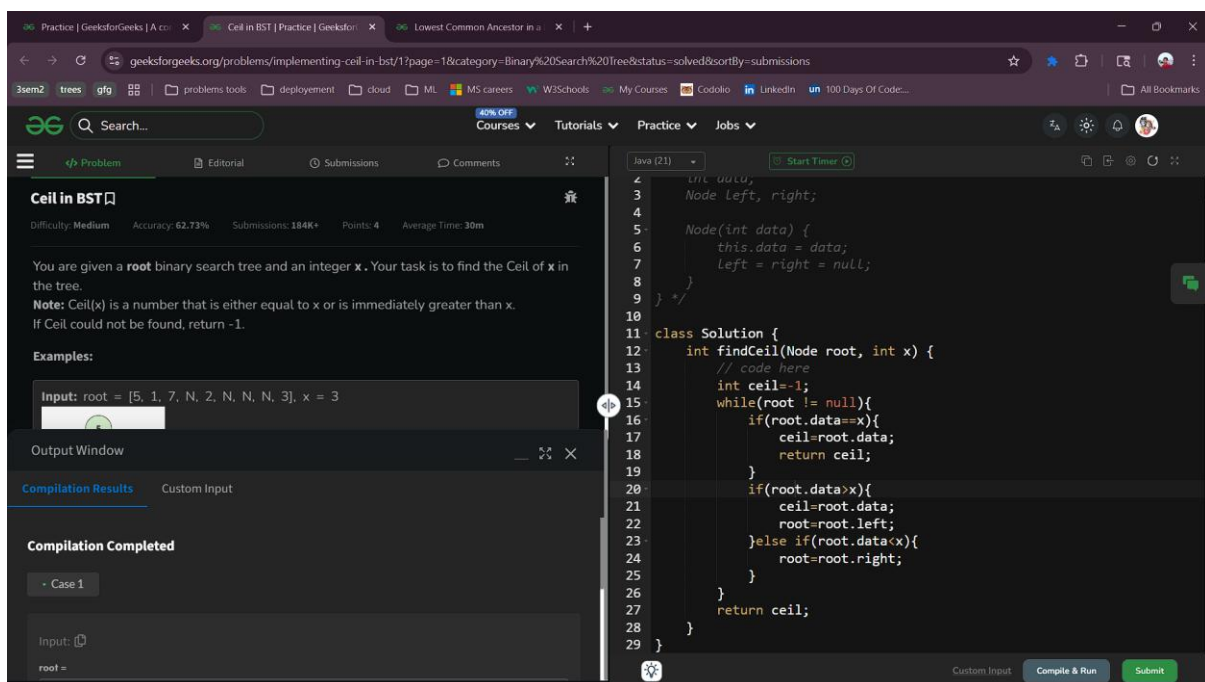
**Aim :** You are given a root binary search tree and an integer  $x$  . Your task is to find the Ceil of  $x$  in the tree.

**Note:** Ceil( $x$ ) is a number that is either equal to  $x$  or is immediately greater than  $x$ .

If Ceil could not be found, return -1.

```
class Solution {
    int findCeil(Node root, int x) {
        // code here
        int ceil=-1;
        while(root != null){
            if(root.data==x){
                ceil=root.data;
                return ceil;
            }
            if(root.data>x){
                ceil=root.data;
                root=root.left;
            }else if(root.data<x){
                root=root.right;
            }
        }
        return ceil;
    }
}
```

**Output :**



## 10.8 Lowest Common Ancestor in a BST

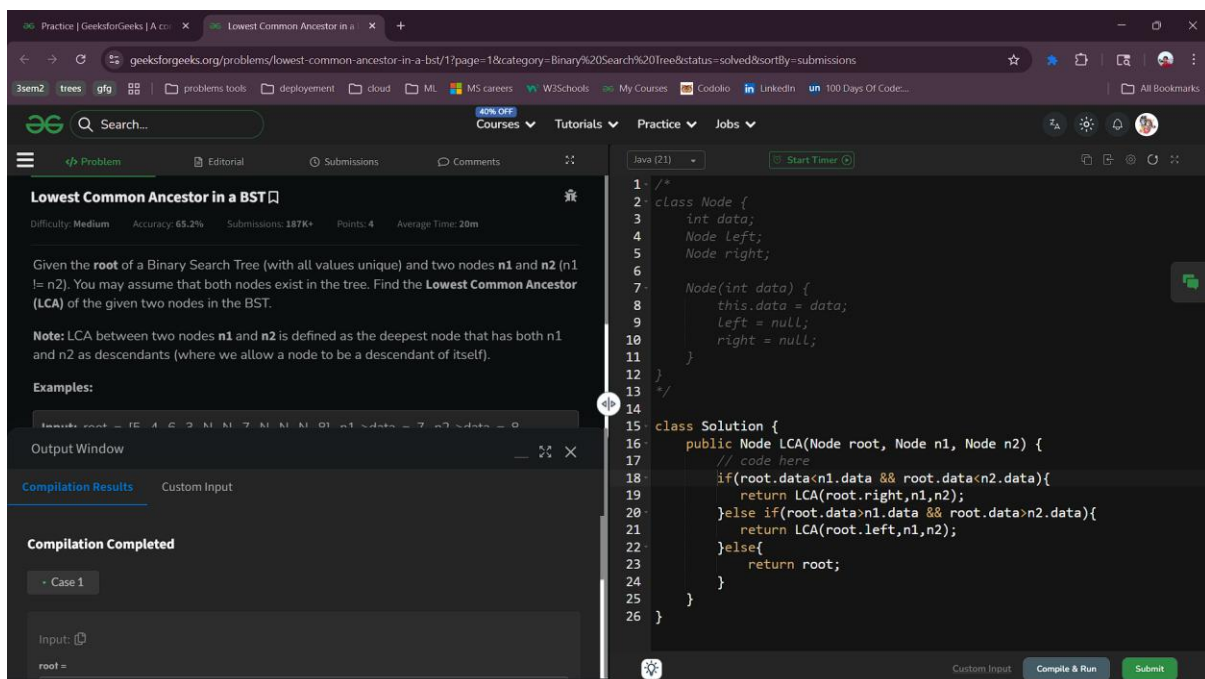
**Aim :** Given the root of a Binary Search Tree (with all values unique) and two nodes  $n1$  and  $n2$  ( $n1 \neq n2$ ). You may assume that both nodes exist in the tree. Find the Lowest Common Ancestor (LCA) of the given two nodes in the BST.

**Note:** LCA between two nodes  $n1$  and  $n2$  is defined as the deepest node that has both  $n1$  and  $n2$  as descendants (where we allow a node to be a descendant of itself).

```
class Solution {
    public Node LCA(Node root, Node n1, Node n2) {
        // code here

        if(root.data < n1.data && root.data < n2.data){
            return LCA(root.right, n1, n2);
        } else if (root.data > n1.data && root.data > n2.data){
            return LCA(root.left, n1, n2);
        } else {
            return root;
        }
    }
}
```

**Output :**



## 11 System Design : Online Shopping Cart

### Case study - 2 - Online Shopping Cart

Design the shopping cart module for an e-commerce platform. The system should allow each user to have a shopping cart with possible products/items and quantities. The system should enable adding, removing and updating items (products) and also quantities in the cart at anytime. At anytime the system should support view cart checkout workflows for the customer and at anytime before completing the transaction he can go back to the shopping cart. The system should calculate the cart total including offers, discounts and taxes. The system should handle item stock checks and edge cases. At the checkout time before completing order service. The system should connect with payment. Now complete functional, nonfunctional req's, architecture diagram, identify core entities and design low level System prototype.

### Functional Requirements

1. Users should be able to add item to cart, remove items from cart, able to update quantity and able to view cart anytime.
2. Cart should be able to support multiple items also.
3. The system should check stock availability before checkout.
4. The system should be able to calculate the final price if there are any discounts (coupon code).
5. The system should also calculate tax (18%) for every order.

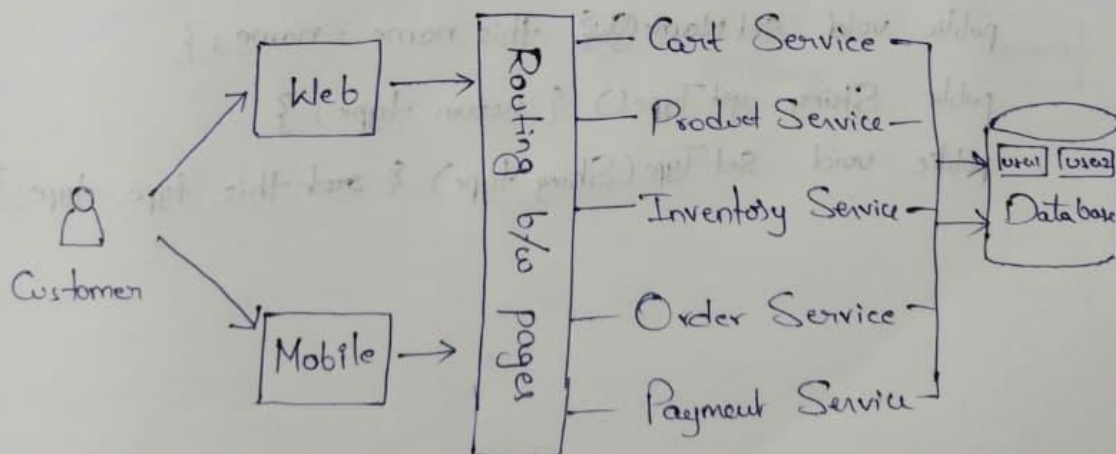
## Non Functional Requirements

1. The system will be able to handle millions of carts for millions of customers.
2. Consistency of stock should be maintained because stock must not be oversold.
3. It's better if we maintain multiple payment types.
4. If the payment is failing then there should be option of retrying.

## Core Entities

User, Product, Cart, Cart Item, Order, Payment, Inventory.

## Architecture



## Prototype Code :

### Product Class :

```
class Product {  
    private int id;  
    private String name;  
    private String type;  
    private double price;  
    private int stock;  
  
    public Product(int id, String name, String type, double price, int  
stock){  
        this.id = id;  
        this.name = name;  
        this.type = type;  
        this.price = price;  
        this.stock = stock;  
    }  
  
    public int getId() { return id; }  
    public void setId(int id) { this.id = id; }  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
    public String getType() { return type; }  
    public void setType(String type) { this.type = type; }  
    public double getPrice() { return price; }  
    public void setPrice(double price) { this.price = price; }
```

```

    public int getStock() { return stock; }
    public void setStock(int stock) { this.stock = stock; }
}

```

### **CartItem Class :**

```

    class CartItem {
        private Product product;
        private int quantity;
        public CartItem(Product product, int quantity) {
            this.product = product;
            this.quantity = quantity;
        }
        public Product getProduct() { return product; }
        public void setProduct(Product product) { this.product = product; }
        public int getQuantity() { return quantity; }
        public void setQuantity(int quantity) { this.quantity = quantity; }
        public double getTotalPrice() {
            return product.getPrice() * quantity;
        }
    }
}

```

### **Cart Class:**

```

import java.util.*;

class Cart {
    private int userId;

```

```
private List<CartItem> items;

public Cart(int userId) {
    this.userId = userId;
    this.items = new ArrayList<>();
}

public int getUserId() { return userId; }

public void setUserId(int userId) { this.userId = userId; }

public List<CartItem> getItems() { return items; }

public void setItems(List<CartItem> items) { this.items = items; }

public void addItem(Product product, int quantity) {
    for (CartItem item : items) {
        if (item.getProduct().getId() == product.getId()) {
            item.setQuantity(item.getQuantity() + quantity);
            return;
        }
    }
    items.add(new CartItem(product, quantity));
}

public void removeItem(int productId) {
    items.removeIf(item -> item.getProduct().getId() == productId);
}

public void updateQuantity(int productId, int quantity) {
    for (CartItem item : items) {
        if (item.getProduct().getId() == productId) {
```

```
        item.setQuantity(quantity);
    }
}

public double calculateTotal() {
    double total = 0;
    for (CartItem item : items) {
        total += item.getTotalPrice();
    }
    return total;
}
```

### **ProductSearch Class :**

```
class ProductService {
    private List<Product> products;

    public ProductService(List<Product> products) {
        this.products = products;
    }

    public List<Product> getProducts() {
        return products;
    }

    public Product[] searchByName(String name) {
```

```

List<Product> result = new ArrayList<>();
for (Product p : products) {
    if (p.getName().equalsIgnoreCase(name)) {
        result.add(p);
    }
}
return result.toArray(new Product[0]);
}

public Product[] searchByType(String type) {
    List<Product> result = new ArrayList<>();
    for (Product p : products) {
        if (p.getType().equalsIgnoreCase(type)) {
            result.add(p);
        }
    }
    return result.toArray(new Product[0]);
}

public Product[] searchByNameAndType(String name, String type)
{
    List<Product> result = new ArrayList<>();
    for (Product p : products) {
        if (p.getName().equalsIgnoreCase(name) &&
            p.getType().equalsIgnoreCase(type)) {
            result.add(p);
        }
    }
    return result.toArray(new Product[0]);
}

```

```

        }
    }
    return result.toArray(new Product[0]);
}
}

```

### **CheckoutService Class :**

```

class CheckoutService {
    public boolean validateStock(Cart cart) {
        for (CartItem item : cart.getItems()) {
            if (item.getQuantity() > item.getProduct().getStock()) {
                return false;
            }
        }
        return true;
    }

    public double applyDiscount(double total) {
        if (total > 1000) return total * 0.9;
        return total;
    }

    public double applyTax(double total) {
        return total * 1.18;
    }

    public void checkout(Cart cart) {
        if (!validateStock(cart)) {

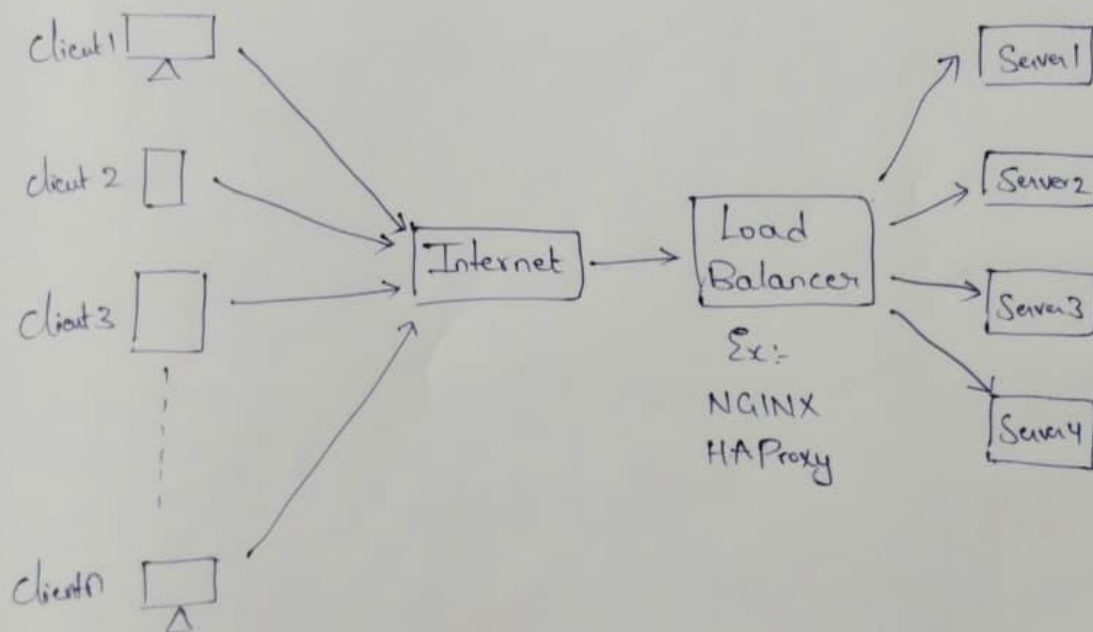
```

```
        System.out.println("Stock insufficient");
        return;
    }
    double total = cart.calculateTotal();
    total = applyDiscount(total);
    total = applyTax(total);
    Payment payment = new Payment(1, total);
    payment.processPayment();
    System.out.println("Payment Status: " + payment.getStatus());
}
}
```

### Edge Cases

1. If user clicks add to cart many times. It should only update one time.
2. If the quantity of product becomes zero then the system removes that product.
3. Product stock = 1 but two users are trying to buy that at same time. (Use Locks)

### Load Balancing



## Caching :

Storing a copy of data which is accessed frequently in RAM, so that retrieval of data will be faster.

## What to Cache in Online Shopping System

1. Storing the cart items in cache will be better.
2. User details like address, personal details.
3. Images of the most popular products should be stored in browser cache.